

Walchand College of Engineering, Sangli

(Government aided Autonomous Institute)

Department of Computer Science and Engineering

TY CSE Mini Project 1(7CS345)

Report on

PromptOpt: Token-Efficient Prompt Optimization for LLMs

Submitted by

Piush U. Gogi	**255200005**
Nirzara A. Manade	**255200006**
Siddhi S. Bolaikar	**255200008**

Under the guidance of

Prof. A. S. Pawar

Department of Computer Science and Engineering,
Walchand College of Engineering, Sangli

2026 - 2027

Walchand College of Engineering, Sangli
(Government aided Autonomous Institute)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Project Report entitled, **"PROMPTOPT: TOKEN-EFFICIENT PROMPT OPTIMIZATION FOR LLMS"** submitted by **Piush U. Gogi, Nirzara A. Manade and Siddhi S. Bolaikar**, to Walchand College of Engineering Sangli, India, is a record of bonafide project work of course **Mini Project-I (7CS345)** carried out by them under my/our supervision and guidance and is worthy of consideration for the award of the degree of Bachelor of Technology in Computer Science & Engineering of the Institute.

Prof. A. S. Pawar	**Mr. H.R. Chandoba**		**Dr. A.R. SURVE**
Guide	**Mini Project Coordinator**	**External Examiner**	**Head Of Department**
Department of Computer Science and Engineering, Walchand College of Engineering, Sangli	Department of Computer Science and Engineering, Walchand College of Engineering, Sangli		Department of Computer Science and Engineering, Walchand College of Engineering, Sangli

Acknowledgement

We express our sincere gratitude to our guide, Prof. A. S. Pawar, for constant guidance, timely reviews and for pushing us to measure every claim instead of assuming it. The suggestions on evaluation and on the validation of our dataset shaped the way this project was carried out.

We thank Mr. H. R. Chandoba, Mini Project Coordinator, for organising the mini-project reviews, and Dr. A. R. Surve, Head of the Department of Computer Science and Engineering, for providing the facilities and the environment to work on this project. We also thank the faculty member who reviewed a sample of our dataset, and the students who answered our customer survey.

Finally, we thank Walchand College of Engineering, Sangli, and our families and friends for their support throughout the project.

Declaration

We hereby declare that the work presented in this project report titled **"PROMPTOPT: TOKEN-EFFICIENT PROMPT OPTIMIZATION FOR LLMS"** submitted by us in the partial fulfillment of the requirement of the award of the degree of **Bachelor of Technology (B. Tech)** in the **Department of Computer Science & Engineering, Walchand College of Engineering, Sangli** is an authentic record of our project work carried out under the guidance of Prof. A. S. Pawar.

Date:

Place: Sangli

(Signature)

Piush U. Gogi
(255200005)
Nirzara A. Manade
(255200006)
Siddhi S. Bolaikar
(255200008)

Table of Contents

Abstract **7**

1 Introduction and Related work **8**

1.1 Background 8
1.2 Related work 8
1.3 Gap analysis 9

2 Problem statement **10**

3 Objectives **11**

4 Methodology **12**

4.1 Dataset 12
4.2 Stage A: feature detection 12
4.3 Stage B: rule-based optimization (B01-B15) 13
4.4 Stage C: LoRA fallback and the B-to-C contract 13
4.5 IR and renderers 14
4.6 Image mode, attachments, Compare 14
4.7 Technology stack 15

5 Project diagrams **16**

5.1 Transaction modules and end-to-end flow 16
5.2 Database design 17
5.3 Evaluation flow 19

6 Test plan **20**

6.1 Unit testing 20
6.2 Integration and system testing 22
6.3 Test cases 22

7 Results **25**

a. Results with screenshot and description 25
b. Comparative analysis 27
c. Performance measures in tabular form and graphs 28
d. Result analysis 30
e. GUI images / screenshots 31
f. Screenshot of GitHub project upload 37
g. Use of Git and GitHub 39
h. Challenges faced and how we solved them 40

8 Conclusion **41**

Limitations of proposed solution 41
Future scope / possible extension to proposed project 41

9 References **42**

10 Scholarly contribution **43**

11 Customer Connect **44**

List of Figures

Figure 1: Dataset pipeline (Dolly-15k and CodeAlpaca-20k to PromptOpt Dataset v1.2)....12
Figure 2: Stage A: feature detection....12
Figure 3: Stage B rule order....13
Figure 4: B-to-C contract: routing, validation and fallback....14
Figure 5: Stage C training pipeline....14
Figure 6: System architecture....16
Figure 7: Module diagram (core and supporting modules, data layer)....16
Figure 8: Transaction flow of one request: submit, optimize, store, Compare, History, retention....17
Figure 9: Entity-relationship diagram (10 tables)....18
Figure 10: Evaluation flow: splits, variants, metrics....19
Figure 11: Automated tests per module (pytest, 798 in total, all passing)....21
Figure 12: Optimize screen: Stage A, Stage B rules with before/after, Stage C, rendered prompt....26
Figure 13: Compare: original vs optimized prompt on the same real model....27
Figure 14: Mean total tokens per prompt by category, vague vs optimized (test split)....28
Figure 15: Benchmark split: quality, task success and total tokens....28
Figure 16: Correctness suite: correct answers per category, vague vs optimized....30
Figure 17: Stage C routed: ambiguous reference sent to Stage C; its answer failed validation, so Stage B's result is kept....31
Figure 18: Test suite page: correctness suite summary and case grid....32
Figure 19: Test suite: one case with gold answer, both prompts and both answers....32
Figure 20: Results page: headline numbers parsed from FINAL_RESULTS.md....33
Figure 21: History: search, view and delete past prompts (30-day retention)....34
Figure 22: Image mode: user's words kept, missing attributes as clickable suggestions....35
Figure 23: How it works page....36
Figure 24: Status page: providers, 24-hour usage, Stage C device, sandbox....37
Figure 25: GitHub repository piush365/PromptOpt....38
Figure 26: GitHub release v1.0 with the Stage C adapter and Stage A index as assets....38
Figure 27: Branches: main (default), ui-v2, correctness-suite....39
Figure 28: Commit timeline per ISO week (all branches)....40

Acronyms

API	Application Programming Interface
CLIP	Contrastive Language-Image Pre-training
CI	Confidence Interval
ER	Entity-Relationship
GPU	Graphics Processing Unit
IR	Intermediate Representation
JSON	JavaScript Object Notation
LLM	Large Language Model
LoRA	Low-Rank Adaptation
ORM	Object-Relational Mapping
PEFT	Parameter-Efficient Fine-Tuning
PII	Personally Identifiable Information
QA	Question Answering
SD	Stable Diffusion
SRS	Software Requirements Specification
UI	User Interface
bwrap	bubblewrap (Linux sandboxing tool)

Abstract

People write prompts for large language models (LLMs) the way they type a search query: short, vague and without saying what the answer should look like. The model then guesses, often producing long, unrequested answers that cost output tokens, latency and money and are harder to use. PromptOpt is a lightweight, self-hosted system that rewrites such a prompt into a clear, structured and token-efficient one for the model the user actually uses (GPT, Gemini or Claude), explains every change, and measures whether the rewrite helps.

The pipeline has three stages. Stage A detects what a prompt is missing (task category, output format, length, tone, audience, programming language, filler, ambiguous references) using spaCy, regular expressions and a sentence-embedding classifier. Stage B applies fifteen deterministic, individually unit-tested rules (B01-B15, including one rule per attachment type) and logs every change. Stage C, a Qwen2.5-0.5B-Instruct model with a LoRA adapter trained on a 4 GB laptop GPU, is called only for what the rules cannot resolve (6.4% of test prompts) and its answer is validated before use. The result is a model-agnostic intermediate representation rendered per target LLM. A FastAPI back end with a React user interface offers optimization, a side-by-side Compare on a real model, a history with 30-day retention, an image-generation mode and an evaluation dashboard; all data goes through one repository module with one database commit per request.

On a held-out test split of 482 prompts, optimized prompts reduce total tokens (input + output) by 39.2% (95% CI 35.9-42.5%) on gpt-oss-120b; the output format is stated in 95.0% of prompts instead of 3.1%; on the benchmark split quality rises from 8.2 to 9.0 and task success from 67% to 85%. On a new suite of 50 hand-written cases with verifiable answers, optimized prompts were correct in 42 of 50 cases versus 37 of 50 for the vague prompts while using 29.6% fewer tokens. The system is covered by 798 automated unit and integration tests and 13 browser end-to-end tests, all passing.

Keywords: prompt optimization, large language models, token efficiency, rule-based NLP, LoRA, FastAPI.

1. Introduction and Related work

1.1 Background

Large language models such as GPT, Gemini and Claude are now used daily by students, developers and small teams. Their cost and speed depend on tokens: every word of the prompt (input) and every word of the answer (output, including hidden reasoning tokens) is counted and billed, and free tiers stop working once a daily token budget is used up. In our customer survey (Chapter 11), most respondents reported running out of tokens quickly and having to ask more than once to get the answer they wanted.

A large part of this waste comes from the prompt itself. A prompt such as "write code to get all permutations of a string" does not say which language to use or what the answer should contain, so the model writes an explanation, several versions and example runs. In our measurements the same model answered that prompt with 768 tokens; the optimized prompt, which adds "Use Python" and "Return only the code, in a single code block", needed 248 tokens (-67.7%) and passed the same 6 of 6 tests.

1.2 Related work

- Automatic prompt optimization. Ramnath et al. [1] survey the field: methods such as APE [4] and OPRO [5] search over candidate instructions with an LLM and score each on a labelled development set. This works for a fixed task but costs many LLM calls per task, does not work per user prompt and is not explainable to the user.
- Cost-aware optimization. CAPO (Zehle et al. [3]) adds prompt length to the objective of an evolutionary search, but still optimizes a task prompt with many LLM evaluations; it is too expensive for a lightweight, self-hosted tool.
- Prompt compression. LLMLingua [6] removes input tokens. Our measurements show the opposite lever: the optimized prompt is slightly longer, and the saving comes from shorter answers. Input-only token counts miss this effect.
- Format sensitivity. Ngweta et al. [2] show that LLM output quality changes with prompt formatting alone. This motivates rendering the same content in the style each model family prefers (XML tags for Claude, Markdown sections for GPT, plain labels for Gemini).
- Parameter-efficient fine-tuning. LoRA [7] trains small low-rank adapters on top of a frozen model, which makes fine-tuning a 0.5-billion-parameter model (Qwen2.5 [10]) possible on a 4 GB laptop GPU.

1.3 Gap analysis

Table 1: Gap analysis: existing prompt tools vs PromptOpt (from the LA-1 review)

Feature	PO.tools	Aiven	SiteGPT	BenchLM	PromptOpt
Rewrite prompts	Yes	Yes	Yes	Yes	Yes
Goal / framework customization	Limited	Yes	Yes	Limited	Yes
Preserve constraints	No	-	-	Yes	Yes
LLM output evaluation	No	No	No	No	Yes
Token efficiency report	-	Yes	-	-	Yes
Multi-stage pipeline	No	No	No	No	Yes
Access	Free	Free	Free	Limited	Self-hosted

Gap addressed: a per-prompt, offline, rule-first optimizer with a small LoRA fallback, model-specific rendering, and an honest evaluation of net tokens (input + output, including hidden reasoning tokens) on a held-out test split, with every change logged and explained to the user.

2. Problem statement

Users often write prompts that are vague, verbose or missing key details (output format, length, audience, programming language, which text is meant by "this"). This wastes tokens, raises API costs, increases latency and gives inconsistent results, and there is no lightweight, self-hosted tool that fixes a single user prompt immediately, explains what it changed, tailors the result to the target LLM, and proves with measurements that the rewrite actually helps.

Formally: given a user prompt p, a target LLM t in {GPT, Gemini, Claude}, an optional category c and an optional attachment type a, produce an optimized prompt p' rendered for t such that (i) the user's intent is preserved, (ii) missing format and constraints are stated, (iii) every change is logged, and (iv) the total tokens (input + output) of answering p' are lower than for p without lowering answer quality or correctness, as measured on held-out data.

3. Objectives

1. Detect what a prompt is missing: task category, output format, constraints (length, tone, audience, language), filler and redundancy, and ambiguous references (Stage A).
2. Fix the prompt with deterministic, individually testable rules, and use a small fine-tuned model only for what the rules cannot resolve (Stage B and Stage C).
3. Produce one model-agnostic intermediate representation (IR) and render it for GPT, Gemini and Claude, with token counts per target.
4. Support an optional attachment type (image, PDF, PPTX, DOCX, spreadsheet, code file) as a modifier, and a separate image-generation mode.
5. Measure instead of assuming: tokens (input and output), quality, task success, correctness, latency, plus an ablation study, on a held-out test split that is run once.
6. Provide a working web application: optimize, Compare original vs optimized on a real model, history with 30-day retention, privacy (PII scrubbing) and one database transaction per request.
7. Run self-hosted and offline (the optimizer needs no API), and treat the target LLMs as black boxes.

4. Methodology

The project followed a fixed roadmap (recorded in the repository's CLAUDE.md): dataset, rule-based optimizer, evaluation harness, ablation, IR and rendering, LoRA fallback, web application. Development decisions were made on the validation split only; the code used for the final numbers was tagged frozen-for-test before the test split was run once (tag final-for-test).

4.1 Dataset

PromptOpt Dataset v1.2 final contains 5,184 degraded -> optimized prompt pairs in five categories, built from Dolly-15k [8] (closed_qa, information_extraction, classification, summarization) and CodeAlpaca-20k [9] (coding). Splits share no instruction: train 4,509, validation 149, test 482, benchmark 44. Three team members rated 170 rows each (90 shared rows, five yes/no questions): 330 rows validated, 295 accepted, 35 rejected and removed; the faculty reviewer accepted 19 of 20 rows. An LLM-assisted filter removed 9 more rows that added facts, leaked the answer or changed the task.

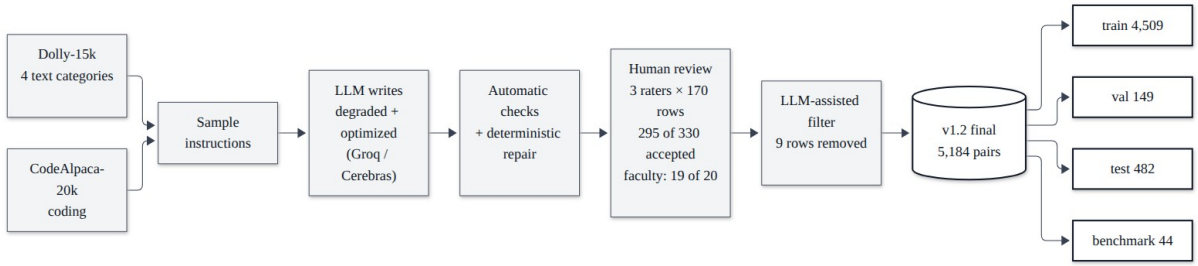


Figure 1: Dataset pipeline (Dolly-15k and CodeAlpaca-20k to PromptOpt Dataset v1.2)

4.2 Stage A: feature detection

Stage A turns the raw prompt into a PromptFeatures object: the task category with a confidence (a sentence-transformer embedding classifier with a linear head trained on the train split, keyword cues as fallback), missing output format (detector A02), missing constraints (A03: length, tone, audience, language), filler and repetition (A04) and ambiguous references such as "summarize this" without any text (A05, spaCy + regular expressions).

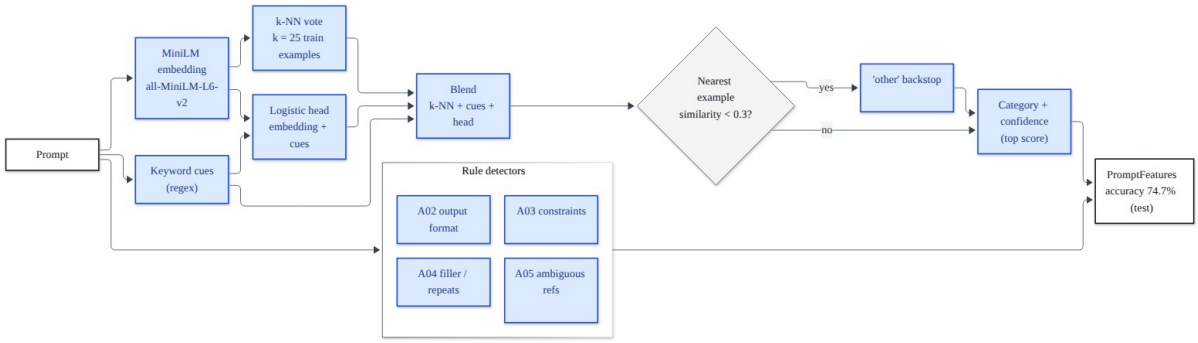


Figure 2: Stage A: feature detection

4.3 Stage B: rule-based optimization (B01-B15)

Table 2: Stage B rule catalogue (backend/app/db/seed.py)

Code	Rule	What it does
B01	Remove filler	Drops words like "hey can you just" that cost tokens and say nothing
B02	Remove duplicates	Removes repeated sentences and doubled words
B03	Output format	Adds the category's default format (e.g. "Return only the code, in a single code block")
B04	Length	Adds a length limit where it matters (e.g. "Keep it under 100 words")
B05	Programming language	Coding: "Use Python." or "Keep the language of the given code"
B06	Label set	Classification: states the allowed labels explicitly
B07	Structure	Moves pasted data into its own block; capital letter and full stop
B08	Group fallback	Low confidence but QA/extraction/summary group sure: "Answer from the provided text in at most three sentences"
B09-B15	Attachment modifier	One rule per attachment type: image, PDF, PPTX, DOCX, other, spreadsheet, code

Category-specific rules (B03-B06) fire only when Stage A's confidence is at least 0.6; below that, B08 handles the closed_qa / information_extraction / summarization group without choosing one of them. Every rule is a separate pure function with its own unit tests, and every change is logged with the text before and after it, which the user interface shows step by step.

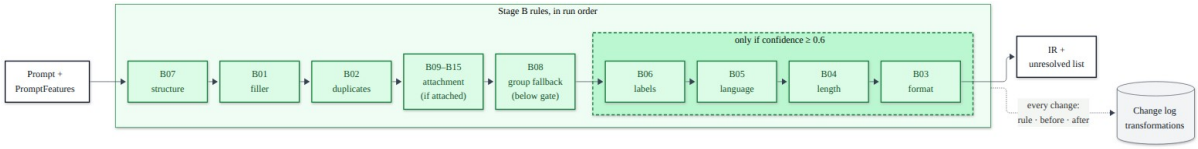


Figure 3: Stage B rule order

4.4 Stage C: LoRA fallback and the B-to-C contract

Stage C is Qwen2.5-0.5B-Instruct with a LoRA adapter (rank 16), trained locally on an RTX 3050 laptop GPU (4 GB, bf16, micro-batch 1 x 16 gradient accumulation, gradient checkpointing) on 4,456 training examples; best validation loss 0.7047 at step 550, early stop at step 700 after 51 minutes. It is called only when Stage B leaves the task category or an ambiguous reference unresolved. It must return JSON with exactly the requested keys; the answer is checked with Stage A's own detectors and rejected, keeping Stage B's result, if invalid. Stage C's category is only a suggestion: the user interface asks the user and pre-selects it.

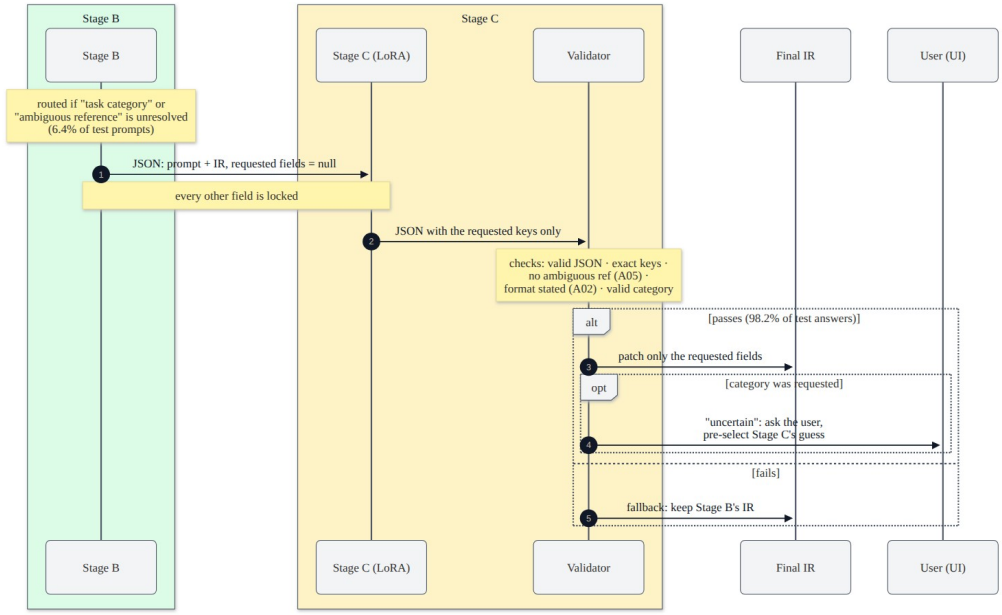


Figure 4: B-to-C contract: routing, validation and fallback

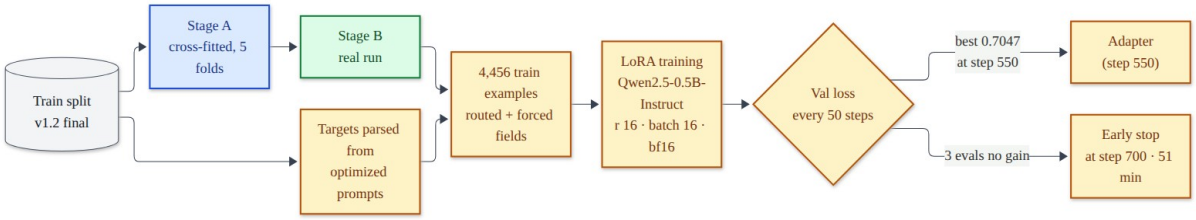


Figure 5: Stage C training pipeline

4.5 IR and renderers

The result of Stages A-C is a model-agnostic intermediate representation (task, context, constraints, requirements, output format, attachment, target LLM, category source, unresolved fields). Renderers lay the same content out per target: Claude gets XML tags with the context first, GPT gets "### Task / Context / Constraints / Output format" sections, Gemini gets plain labels with the instruction first. Tests parse every rendering back to check that no field is lost. Input tokens are counted per target: GPT exactly with tiktoken (o200k_base), Claude and Gemini approximately (characters / 4, labelled as such).

4.6 Image mode, attachments, Compare

- Image-generation mode is a separate mode the user selects (never auto-detected). Version 2 keeps the user's words first and offers every missing attribute (style, lighting, palette, aspect ratio, ...) as a clickable suggestion instead of inserting defaults.
- Attachments are modifiers, not categories: rules B09-B15 add one requirement per attachment type and fire only when an attachment type is given.
- Compare sends the original and the optimized prompt to the same real model (gpt-oss-120b on Groq

or Cerebras; Gemini when a key is set) at temperature 0 and shows both answers, tokens, latency, sandbox test results for coding prompts and an optional blind judge score, with honest labels when a stand-in model answers.

4.7 Technology stack

Table 3: Technology stack

Layer	Technologies
Front end	React 19 + TypeScript, Vite, Tailwind CSS, Radix UI, Recharts; Playwright for end-to-end tests
Back end	Python 3.14, FastAPI, Uvicorn, Pydantic
Database	SQLAlchemy 2.0 ORM; SQLite (development), PostgreSQL (deployment)
NLP / features	spaCy, regular expressions, Sentence-Transformers (all-MiniLM-L6-v2), pyspellchecker
Model / fine-tuning	Hugging Face Transformers, PEFT (LoRA), PyTorch (CUDA), Qwen2.5-0.5B-Instruct
Tokenization	tiktoken (o200k_base)
LLM providers (evaluation)	Groq and Cerebras free tiers (gpt-oss-120b, qwen3.8-27b judge); Gemini optional
Testing / tooling	pytest (798 tests), Playwright (13 tests), bubblewrap sandbox, Git + GitHub

5. Project diagrams

This chapter collects the architecture, module, transaction-flow, database and evaluation diagrams. The Mermaid sources of the system diagrams are committed in docs/diagrams/.

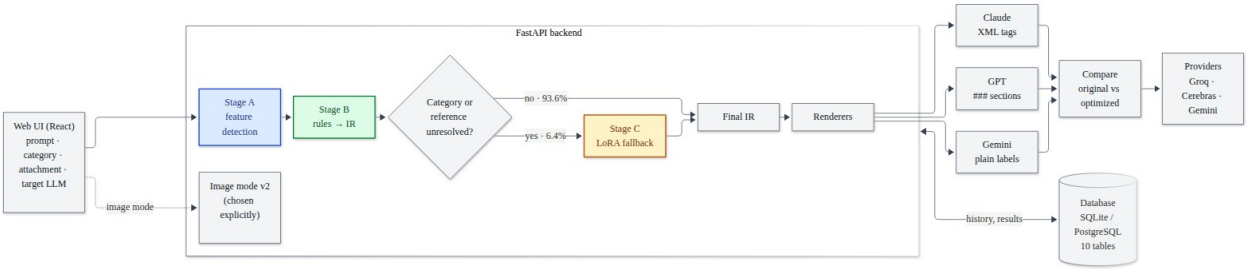


Figure 6: System architecture

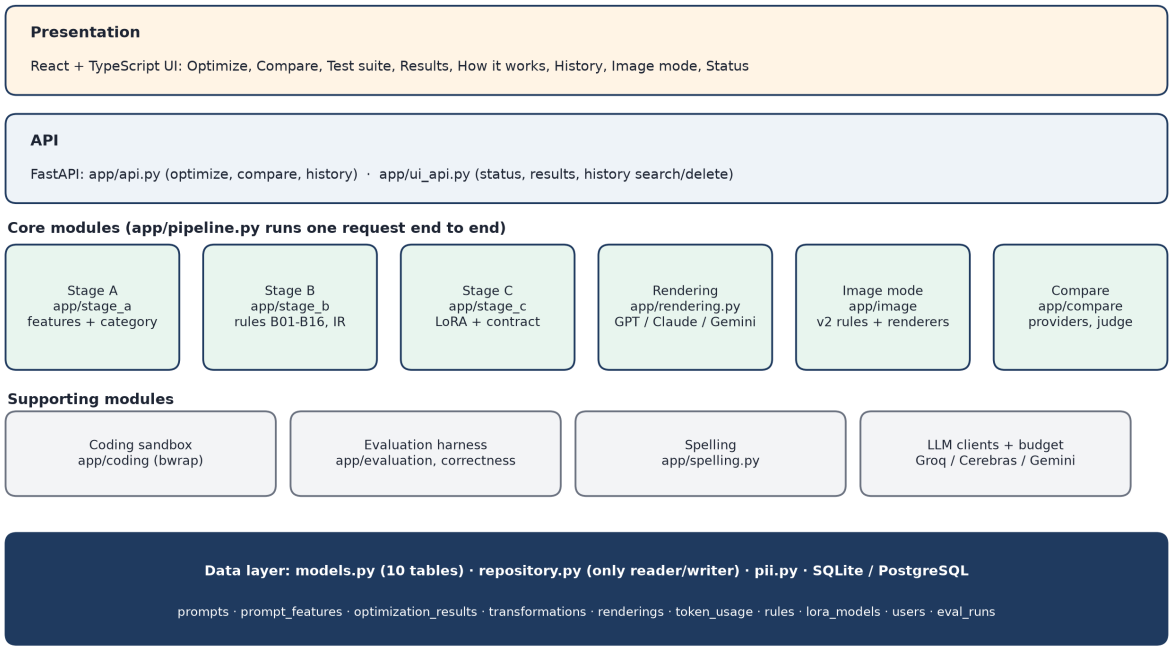


Figure 7: Module diagram (core and supporting modules, data layer)

5.1 Transaction modules and end-to-end flow

One optimization request is one database transaction. The FastAPI endpoint first deletes expired prompts (retention), then app/pipeline.py runs the stages and stores their results only through app/db/repository.py, whose functions flush (so that ids exist) but never commit. The endpoint commits once at the end, so a failure half-way leaves nothing behind.

1. The user submits the prompt, target LLM, category (auto or one of five) and optional attachment type (POST /api/optimize).
2. create_prompt strips e-mail addresses and phone numbers (PII) and sets expires_at = created_at +

30 days (table prompts).

3. Stage A runs; save_features stores the detected features (table prompt_features).
4. Stage B (and Stage C if routed) runs; save_optimization stores the IR and the optimized text (table optimization_results) and one row per change (table transformations, linked to the rules table).
5. The IR is rendered for the targets; save_renderings and save_token_usage store them (tables renderings, token_usage).
6. db.commit() is called once; the response returns features, rules fired, renderings and token counts.
7. Compare (POST /api/compare) runs the scrubbed original and the optimized prompt on the same model and stores the run; History (GET /api/ui/history) lists, searches, shows and deletes past prompts (cascade).
8. Retention: expired prompts and everything derived from them are deleted automatically before every request that stores, reads or lists prompts, and on demand with python -m app.init_db --purge.

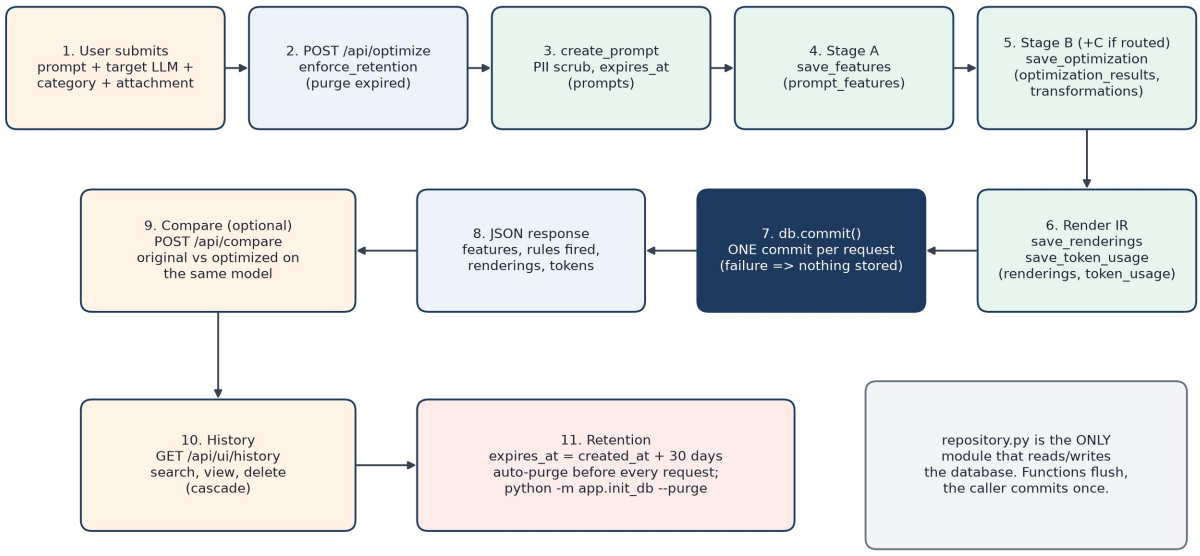


Figure 8: Transaction flow of one request: submit, optimize, store, Compare, History, retention

5.2 Database design

The database has ten tables (app/db/models.py). The prompt-side tables cascade on delete so that the retention purge removes everything derived from an expired prompt; evaluation_runs is deliberately not linked to prompts so that benchmark results are never purged. Constraints are enforced in the schema (e.g. confidence between 0 and 1, a Stage B transformation must reference a rule, unique rendering per result and target).



Figure 9: Entity-relationship diagram (10 tables)

Table 4: Tables written in one optimization transaction

Table	Written by	Content
prompts	create_prompt	PII-scrubbed original text, pii_redactions, created_at, expires_at
prompt_features	save_features	task type, confidence, missing constraints, redundant phrases, ambiguous refs
optimization_results	save_optimization	IR (JSON), optimized text, confidence, used_lora, latency
transformations	save_optimization	step number, stage (B/C), rule_id, text before and after
renderings	save_renderings	rendered prompt per target LLM
token_usage	save_token_usage	original/optimized input (and output) tokens per target, tokenizer, cost

5.3 Evaluation flow

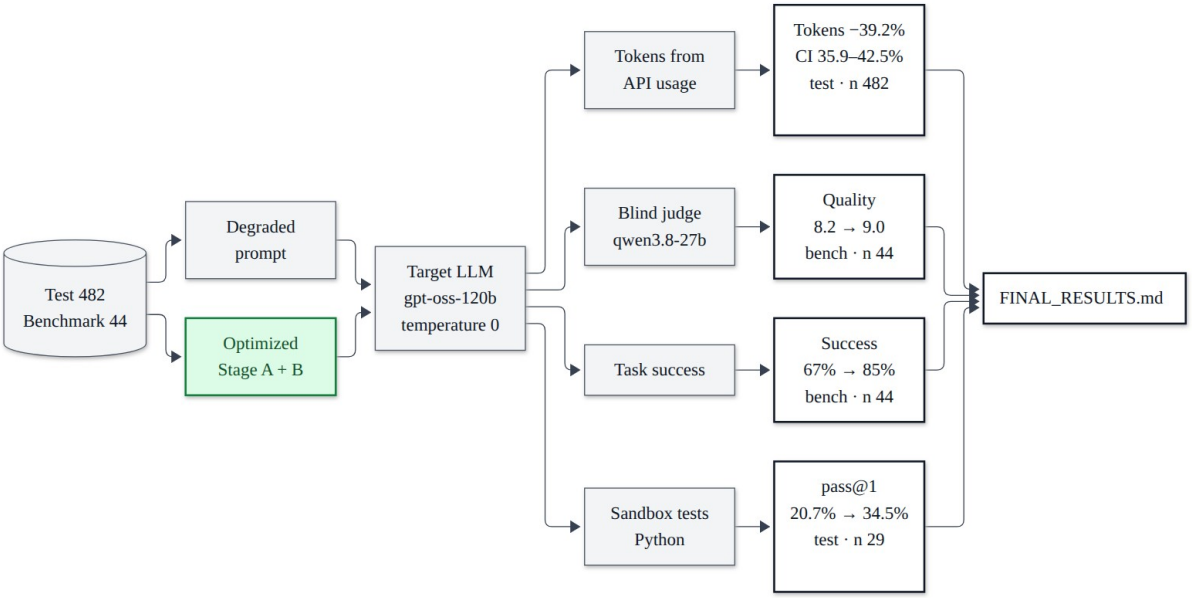


Figure 10: Evaluation flow: splits, variants, metrics

6. Test plan

Testing is automated. Unit and integration tests run with pytest inside backend/ and need no network: LLM providers are faked or replayed from recordings, and every database test runs on in-memory SQLite (and on PostgreSQL when TEST_POSTGRES_URL is set). Browser end-to-end tests run with Playwright against the real FastAPI application on a throw-away database. System tests run the full pipeline on the dataset splits and on a hand-written correctness suite. Manual testing covered the demo flow and human review of the dataset and of the correctness-suite gold answers (50 of 50 confirmed).

Table 5: Types of testing and tools

Level	Tool	What is tested	Result (run 14 Oct 2026)
Unit	pytest	every Stage A detector, every Stage B rule, Stage C contract, renderers, DB, sandbox, clients	798 passed (incl. integration)
Integration	pytest + FastAPI TestClient	API end to end, pipeline storage, Compare with recorded answers, retention	included above
System (UI)	Playwright + Chrome	optimize, compare, test suite, history, theme, image mode, tour, console errors	13 / 13 passed
Regression	python -m app.freeze_check	Stage A+B on 482 test prompts vs tag frozen-for-test	byte-identical
System (pipeline)	evaluation harness	482-prompt test split, 44-prompt benchmark, ablation	Chapter 7
Acceptance	correctness suite	50 hand-written cases with verifiable answers	vague 37/50, optimized 42/50
Security	bubblewrap sandbox tests	no network, read-only system, memory/time/fork limits	passed

6.1 Unit testing

Run: cd backend && source .venv/bin/activate && python -m pytest -q. Result on 14 October 2026: 798 passed in 24.6 s. Parametrized tests run several cases per test function, hence 798 tests.

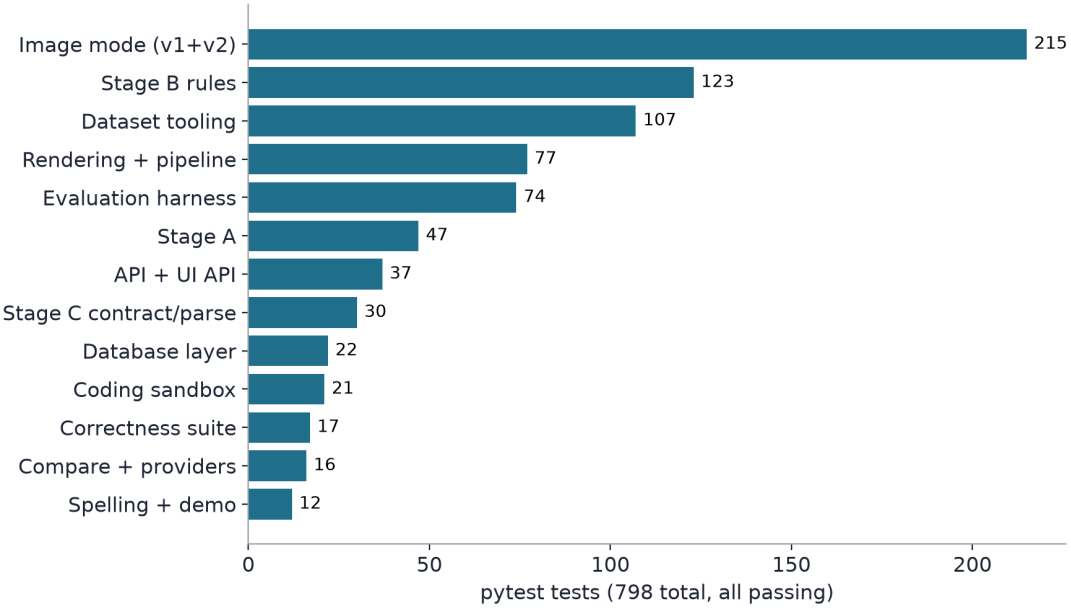


Figure 11: Automated tests per module (pytest, 798 in total, all passing)

Table 6: Test groups and what they cover

Test file(s)	Tests	Covers
test_stage_a.py	47	detectors A02-A05 (positive and negative), context detection, embedding classifier, index round trip
test_stage_b.py, test_stage_b_extensions.py	123	every rule B01-B16 separately, rule order, 0.6 gate, B08, routing, ablation switches, attachments
test_stage_c_contract.py, test_stage_c_parse.py	30	requested fields, each validation failure, patching, locked fields, category policy, parser
test_rendering.py	77	round trip of every rendering, order, fences, token counting, pipeline storage incl. Stage C steps
test_db.py	22	schema constraints, cascade, PII scrubbing, retention purge (SQLite and PostgreSQL)
test_api.py, test_ui_api.py	37	endpoints with/without Stage C, image mode, history, 422 errors, retention, password, concurrency
test_compare.py, test_cerebras.py	16	Compare with recorded answers, stand-in labels, budgets, Gemini/Cerebras clients
test_coding.py	21	sandbox (timeout, memory, network, host files, fork limit, refusal without bwrap), naming discovery
test_image.py, test_image_v2.py	215	image attribute detectors, v1 rules and renderers, v2 suggestions and conflicts
test_evaluation.py, test_correctness.py, test_rule_accuracy.py, test_tokens_eval.py, test_attachment_eval.py	91	judge parsing, task success, resumable harness, scoring per category, McNemar, statistics
dataset tooling (4 files)	107	dataset expansion, repair, validation statistics (kappa, AC1), reference checks
test_spelling.py, test_demo.py	12	typo suggestions and offsets, demo script

Sample of real unit tests (backend/tests):

```
def test_b05_defaults_to_python():
    text = "Write a function that reverses a list."
    assert b.b05_add_language(ir_of(text, "coding"),
                              feats(text, "coding")).constraints == ("Use Python.",)


def test_pii_is_scrubbed_before_storage(db):
    p = repo.create_prompt(db, "mail rahul.k@example.com or call 98765 43210 / 555-123-
4567 about the report")
    db.commit()
    stored = db.scalar(select(Prompt.original_text).where(Prompt.id == p.id))
    assert stored.count("[EMAIL]") == 1 and stored.count("[PHONE]") == 2
    assert p.pii_redactions == 3


def test_blank_prompt_is_rejected_with_422_not_500(client):
    r = client.post("/api/optimize", json={"prompt": "   \n ", "target": "gpt"})
    assert r.status_code == 422 and "empty" in r.json()["detail"]


def test_no_network():   # coding sandbox
    r = run_python({"main.py": "import socket ... socket.create_connection(('1.1.1.1',
53))"})
    assert r.ok and r.stdout.startswith("blocked")
```

6.2 Integration and system testing

- API end-to-end tests (FastAPI TestClient): optimize with and without Stage C, history, Compare with recorded answers, image mode, blank prompts (422), retention before history, PII-scrubbed Compare inputs.
- Freeze check: python -m app.freeze_check recomputes Stage A + B on all 482 test prompts and compares a sha256 over every feature, IR, text, step, routing decision and confidence with tag frozen-for-test. Run on 14 October 2026: BYTE-IDENTICAL (35f7d4dc...).
- Full-pipeline runs on the 482-prompt test split (Stage A, Stage B, Stage C ablation, token evaluation) and the 44-prompt benchmark with a real LLM and a blind judge (Chapter 7).
- Coding sandbox tests: 3-6 assert tests per Python coding item, kept only if they pass on the reference solution, run in a bubblewrap sandbox (no network, read-only system).
- Correctness suite: 50 new hand-written cases (10 per category) with one verifiable answer each; gold answers human-reviewed (50 of 50 confirmed).
- UI end-to-end tests: 13 Playwright tests in frontend/e2e/app.spec.ts (npm run e2e), all passing on 14 October 2026.

6.3 Test cases

The table below lists test cases taken from the real test suite (TC01-TC30) and from the correctness suite (TC31-TC38). The correctness rows report whether the optimized prompt's answer was correct; the FAIL rows are the cases where it was not, including the four where optimization made a correct answer wrong.

Table 7: Test cases (unit, integration, system and correctness suite)

ID	Module	Description	Input	Expected	Actual	Status
TC01	Stage A	Ambiguous reference without text	"summarize this", no context	flagged as ambiguous (A05)	flagged	PASS
TC02	Stage A	No ambiguity when text is given	"summarize this" + context	no ambiguous reference	none	PASS
TC03	Stage A	Embedding classifier falls back to other	prompt unlike any training prompt	category = other	other	PASS
TC04	Stage B	B01 removes filler	"hey can you please summarize the text for me?"	"Summarize the text."	as expected	PASS
TC05	Stage B	B01 keeps filler words that are content	make a button that says "thank you so much"	quoted text unchanged	unchanged	PASS
TC06	Stage B	B05 default language	"Write a function that reverses a list." (coding)	constraint "Use Python."	as expected	PASS
TC07	Stage B	B05 named language left alone	"Write a Rust function to add two numbers."	IR unchanged	unchanged	PASS
TC08	Stage B	B05 keeps language of supplied code	"Edit that function..." + JS code	"Keep the language of the given code"	as expected	PASS
TC09	Stage B	B08 group fallback	"when was it finished" + passage, closed_qa 0.45	adds "Answer from the provided text in at most three sentences."	as expected	PASS
TC10	Stage B	Idempotence	optimized prompt optimized again	nothing new added	nothing added	PASS
TC11	Stage B	Attachment rules never fire without attachment	prompts of every category, no attachment	B09-B15 not fired	not fired	PASS
TC12	Stage C	Invalid answer falls back to Stage B	malformed / wrong-key JSON from the model	fallback, Stage B IR kept	fallback	PASS
TC13	Stage C	Not routed means not called	prompt with nothing unresolved	Stage C not called	not called	PASS
TC14	Rendering	Every field survives every renderer	IR rendered for GPT, Claude, Gemini	parsed back equal to the IR	equal	PASS
TC15	Rendering	Token counts per target	rendered prompt	GPT exact, Claude/Gemini approximate	as expected	PASS
TC16	Database	PII scrubbed before storage	"mail rahul.k@example.com or call 98765 43210 / 555-123-4567"	1 [EMAIL], 2 [PHONE], 3 redactions	as expected	PASS
TC17	Database	Retention purge cascades	one 31-day-old and one fresh prompt	old prompt and all derived rows deleted	1 purged, children cascaded	PASS
TC18	API	Blank prompt rejected	POST /api/optimize, prompt " "	HTTP 422 (not 500)	422	PASS
TC19	API	Expired prompts hidden from History	prompt older than 30 days	deleted before history is listed	deleted	PASS
TC20	API	Models load once under concurrency	parallel optimize requests	one Stage A/C model instance	one instance	PASS
TC21	Sandbox	No network inside the sandbox	code opening a socket to 1.1.1.1	connection blocked	blocked	PASS
TC22	Sandbox	Refuses to run without bwrap	SANDBOX_REQUIRE_ BWRAP=1, no bwrap	refusal, code not run	refused	PASS
TC23	Compare	Both prompts PII-scrubbed	prompt with e-mail + pasted text	provider receives scrubbed text only	scrubbed	PASS
TC24	Compare	Gemini rate limits	per-minute 429 / per-day 429	retry / stop for the day	as expected	PASS
TC25	Image mode	v2 keeps every user word	image prompts (parametrized)	no word lost, no default inserted	as expected	PASS
TC26	System	Freeze check (byte-identical)	482 test prompts, Stage A+B	sha256 equal to frozen-for-test	35f7d4dc... equal	PASS
TC27	System (UI)	Gallery example	Playwright: click example	Stage A, B, C,	shown	PASS

ID	Module	Description	Input	Expected	Actual	Status
		shows every stage		renderings shown		
TC28	System (UI)	History list, search, view, delete	Playwright	all four work	work	**PASS**
TC29	System (UI)	Every page loads without console errors	Playwright, 8 pages	0 console errors	0	**PASS**
TC30	System	Attachment rules, hand-made set	30 prompts with attachments	own rule fires, no other	30/30	**PASS**
TC31	Correctness	cqa-01 hostel fee	"im in 2nd yr, already in boys hostel. if i pay on 18 july how much total??"	80300	optimized 80300 (vague 85300)	**PASS**
TC32	Correctness	ie-10 scholarship renewal documents	"what docs do i need for scholarship renewal, i got it last yr"	exactly 4 documents	4 documents (vague listed extra)	**PASS**
TC33	Correctness	cod-09 fix second_highest	"second highest marks code is wrong sometimes fix it"	all hidden asserts pass	all pass (vague 3/8)	**PASS**
TC34	Correctness	cls-09 attendance status	"mark each student safe warning or detained"	label per student	Python script returned (Stage A said coding 0.84)	**FAIL**
TC35	Correctness	sum-06 IV message summary	"summarize this IV msg in 4-5 lines"	all key facts, <= 90 words	all facts, 92 words	**FAIL**
TC36	Correctness	sum-10 PG rules summary	"summarize pg rules main points under 80 words..."	<= 80 words	83 words (title line)	**FAIL**
TC37	Correctness	cod-10 recipe scaler (Stage C routed)	"write the scale fn for recipe app"	all hidden asserts pass	4/8 asserts	**FAIL**
TC38	Correctness	sum-05 recipe steps (Stage C routed)	"give me short steps for this, im a hostel noob, under 70 words"	all key facts	missed facts (vague also wrong)	**FAIL**

7. Results

a. Results with screenshot and description

Table 8: Headline results (test split, run once on 2 October 2026; correctness suite 10 October 2026)

What	Result
Total tokens, optimized vs vague (482 test prompts, gpt-oss-120b)	-39.2% (95% CI 35.9-42.5%)
Stage A category accuracy (degraded prompts)	74.7% (macro-F1 0.746)
Output format stated: vague -> Stage B	3.1% -> 95.0%
Prompts routed to Stage C	6.4% (31 of 482)
Benchmark (44 prompts, real LLM + blind judge): quality / task success	8.2 -> 9.0 / 67% -> 85%
Stage C answers passing validation: zero-shot base vs LoRA	5.0% vs 98.2%
Stage C on routed prompts: output format stated	22.6% -> 90.3%
Stage C latency, median (laptop RTX 3050 / CPU)	0.74 s / 4.55 s
Coding tests pass@1 strict (29 Python items)	20.7% -> 34.5%
Correctness suite (50 cases): vague vs optimized correct	37/50 vs 42/50, total tokens -29.6%
Attachment rules, hand-made set	30/30 correct

The figure below shows the main screen with the example "write code to get all permutations of a string". Stage A detects the coding category with confidence 1.00 and two missing pieces (no output format, no language); Stage B fires three rules (B07, B05, B03) and shows each change; Stage C is skipped because nothing is unresolved; the rendered GPT prompt grows from 9 to 35 input tokens, together with validated tests.

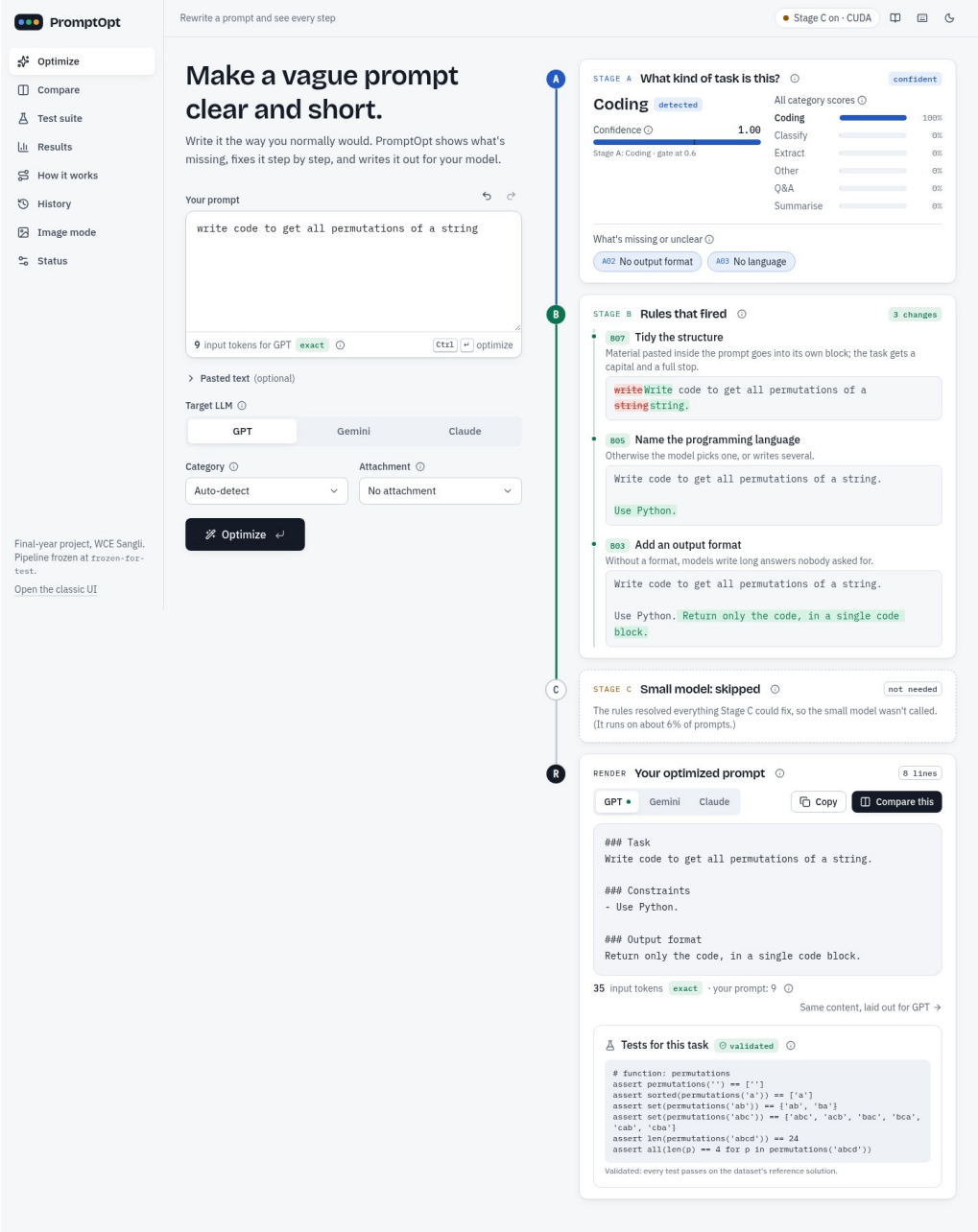


Figure 12: Optimize screen: Stage A, Stage B rules with before/after, Stage C, rendered prompt

The next figure shows the same prompt in Compare on gpt-oss-120b (Groq): total tokens fall from 768 to 248 (-67.7%), output tokens by 79.4%, latency by 78.9%, and both answers pass 6 of 6 sandbox tests.

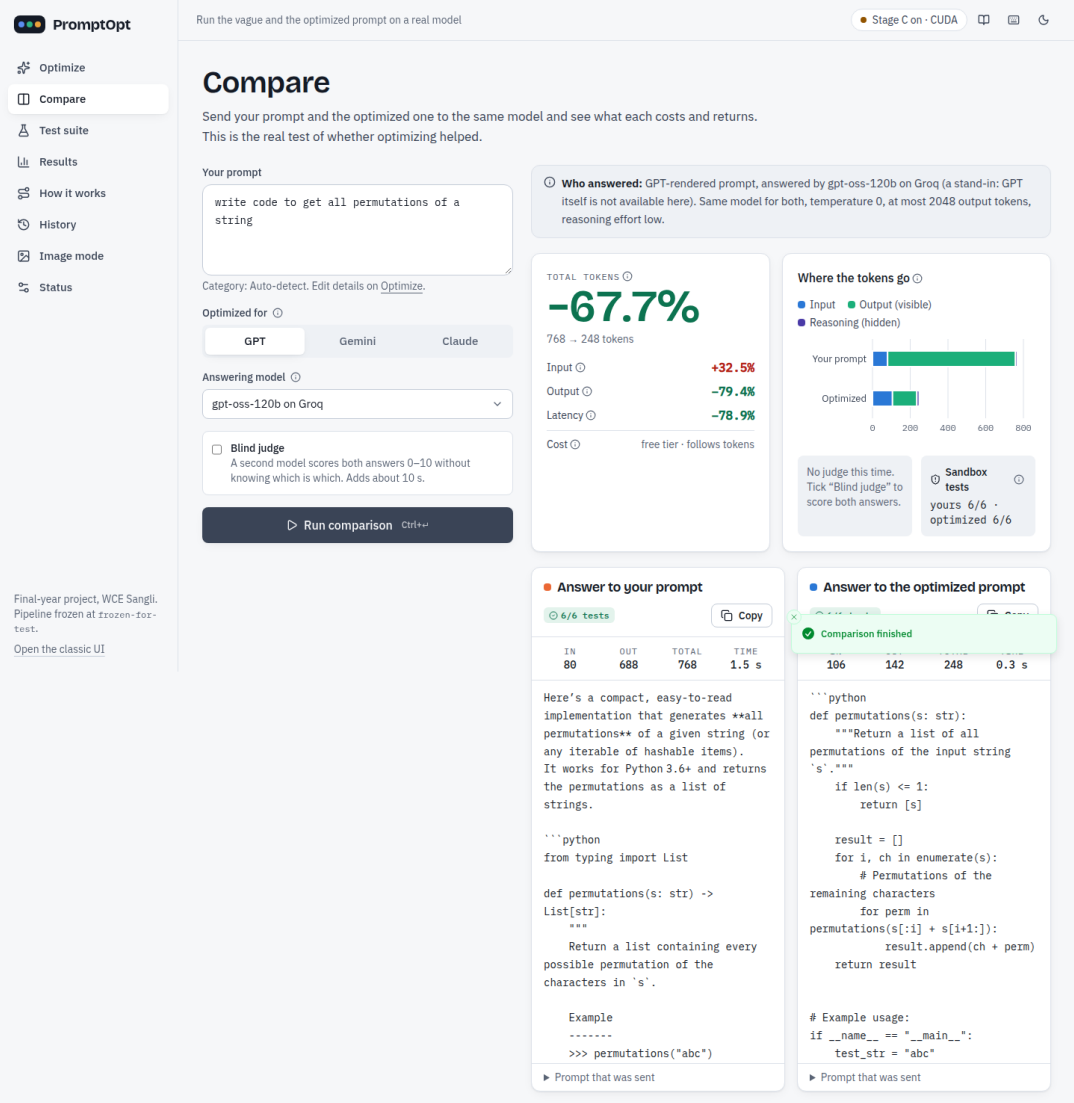


Figure 13: Compare: original vs optimized prompt on the same real model

b. Comparative analysis

The optimized prompt is slightly longer (input 226 -> 241 tokens per prompt), but answers are much shorter (output 682 -> 165), so the total falls in every category. 87 of 482 prompts (18.0%) individually cost more, mostly short-answer information_extraction prompts (35 of 93).

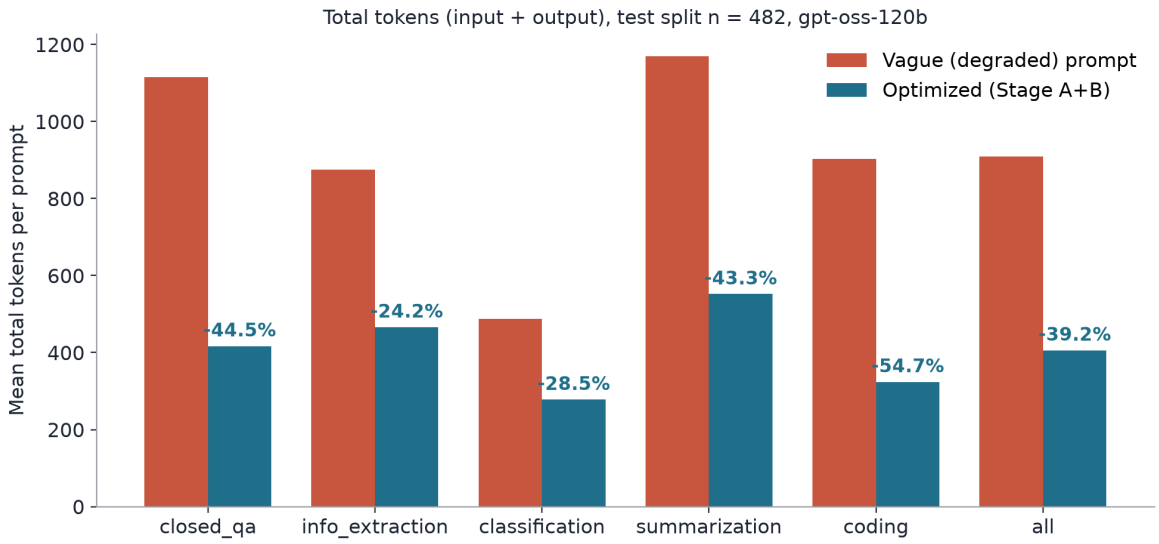


Figure 14: Mean total tokens per prompt by category, vague vs optimized (test split)

Table 9: Token evaluation on the full test split (gpt-oss-120b on Cerebras, temperature 0)

Category	n	Input	Output	Total	Change per prompt [95% CI]
closed_qa	99	279 -> 290	835 -> 127	1115 -> 417	-44.5% [-51.3, -37.6]
information_extraction	93	338 -> 351	538 -> 115	875 -> 466	-24.2% [-32.4, -16.2]
classification	98	98 -> 122	390 -> 156	488 -> 278	-28.5% [-35.8, -21.1]
summarization	94	338 -> 348	831 -> 204	1169 -> 552	-43.3% [-49.4, -37.0]
coding	98	89 -> 102	813 -> 222	903 -> 324	-54.7% [-60.8, -48.0]
all	482	226 -> 241	682 -> 165	909 -> 406	-39.2% [-42.5, -35.9]

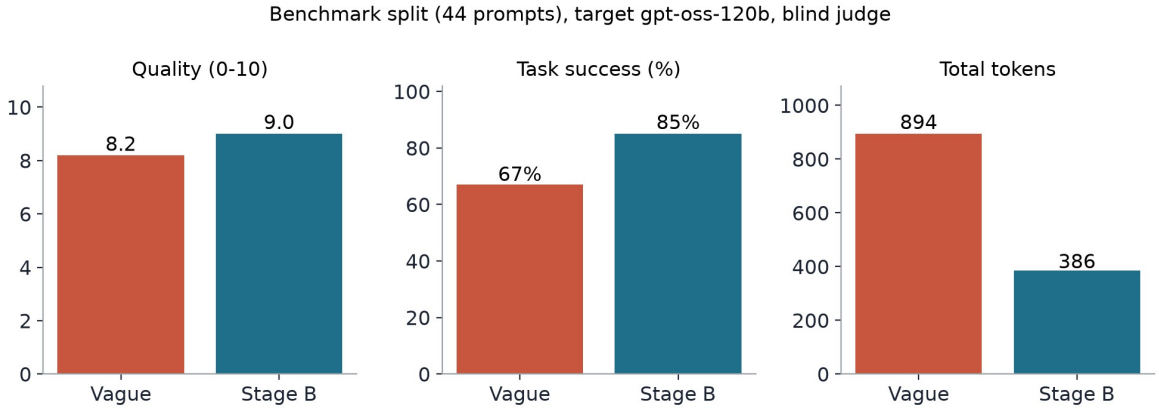


Figure 15: Benchmark split: quality, task success and total tokens

c. Performance measures in tabular form and graphs

Table 10: Stage A per-category results (test, 482 prompts)

Category	n	Precision	Recall	F1
closed_qa	99	56.7%	76.8%	0.65
information_extraction	93	73.2%	55.9%	0.63

Category	n	Precision	Recall	F1
classification	98	93.1%	95.9%	0.94
summarization	94	60.0%	44.7%	0.51
coding	98	99.0%	98.0%	0.98
accuracy (all)	482			74.7%

Table 11: Per-rule accuracy B01-B08 against the dataset's optimized prompts (test)

Sr. No	Rule	Precision	Recall	F1
1	B01 remove filler	1.000	1.000	1.000
2	B03 output format	0.824	0.662	0.734
3	B04 length	0.827	0.315	0.456
4	B05 language	0.925	0.949	0.937
5	B06 labels	0.578	0.649	0.612
6	B07 data moved	0.298	0.824	0.438
7	B08 group fallback	0.732	0.524	0.611
	macro (B01-B08, B07 = data moved)	0.648	0.703	0.684

Table 12: Stage C ablation (test)

Set	System	n	Task intent	Format stated	Fallback
routed	A+B	31	0.869	22.6%	-
routed	A+B+C	31	0.843	90.3%	3.2%
routed	C only	31	0.712	90.3%	6.5%
forced (all)	A+B	482	0.886	95.0%	-
forced (all)	A+B+C	482	0.769	90.2%	2.1%
forced (all)	C only	482	0.765	88.6%	1.9%

Where Stage B routes a prompt, Stage C states a format in 90% instead of 23% at a small loss in task intent; routing everything through Stage C hurts (0.886 -> 0.769), so Stage C stays a fallback for routed prompts only; C-only is the weakest, so the rules are worth keeping in front of the model.

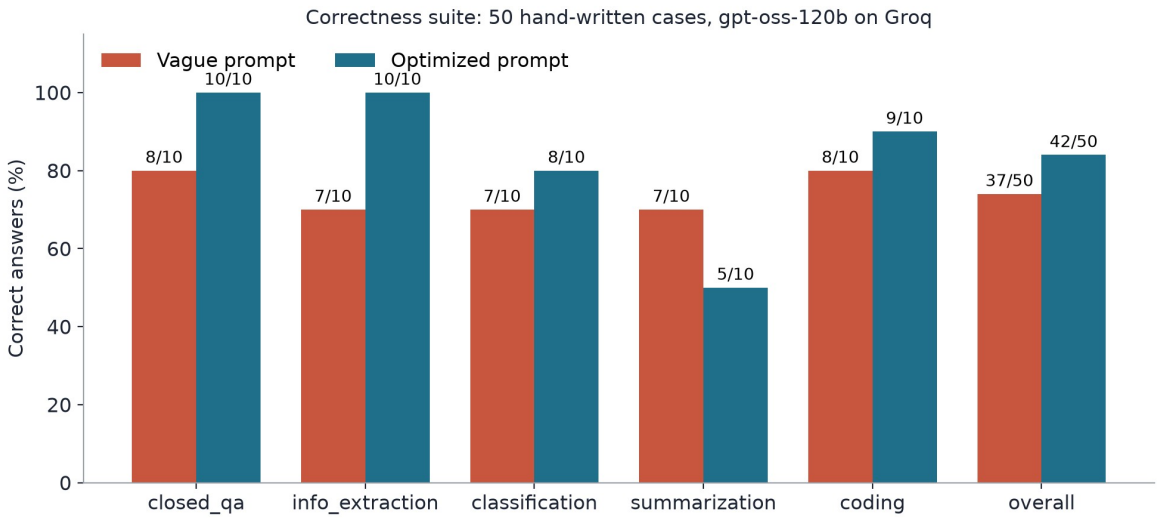


Figure 16: Correctness suite: correct answers per category, vague vs optimized

Table 13: Correctness suite, primary model gpt-oss-120b on Groq (50 cases)

Category	Vague	Optimized	Only opt.	Only vague	Tokens (aggregate)
closed_qa	8/10	10/10	2	0	-22.0%
information_extraction	7/10	10/10	3	0	-27.8%
classification	7/10	8/10	2	1	-6.3%
summarization	7/10	5/10	0	2	+7.9%
coding	8/10	9/10	2	1	-56.0%
overall	37/50	42/50	9	4	-29.6%

d. Result analysis

- Savings come from the answer, not the prompt: a stated format and length stop long, unrequested answers. Prompts whose answer is already short can cost more (e.g. a one-line closed_qa answer: +17.2% in Compare).
- Correctness improved on 9 cases and got worse on 4 (McNemar p = 0.267, not significant at n = 50). On a second provider (Cerebras) the result was 43/50 vs 41/50, so the correctness gain is not robust across providers, while the token saving is (-28.2%).
- Honest failures: cls-09 (Stage A misread an attendance table as code, so the model returned a script instead of labels), sum-06 and sum-10 (length limits exceeded by 2-3 words), cod-10 (a Stage C-routed coding prompt where the model's fraction handling was wrong). Summarization is the weak category (7/10 -> 5/10).
- Stage A is the main weak spot: closed_qa, information_extraction and summarization are hard to tell apart on degraded prompts (F1 0.51-0.65); B08 and asking the user mitigate this.
- Image mode v1 (default keywords) made Stable Diffusion images match the request worse (held-out CLIP 30.78 vs 33.08, p = 0.043); v2 (user's words first, suggestions instead of defaults) shows no

measurable harm (32.77, p = 0.125) and keeps the style in 12 of 12 styled prompts.

e. GUI images / screenshots

Working prototype and demo flow: the app runs as uvicorn app.api:app (FastAPI serving the built React UI) with Stage C on the GPU. Demo order: (1) an example from the gallery on Optimize, showing Stage A, the Stage B rules with before/after text and the GPT/Gemini/Claude renderings; (2) a prompt with an ambiguous reference that is routed to Stage C; (3) Compare on gpt-oss-120b; (4) Image mode with suggestions; (5) History, the Test suite and the Results page. All screens are shown below.

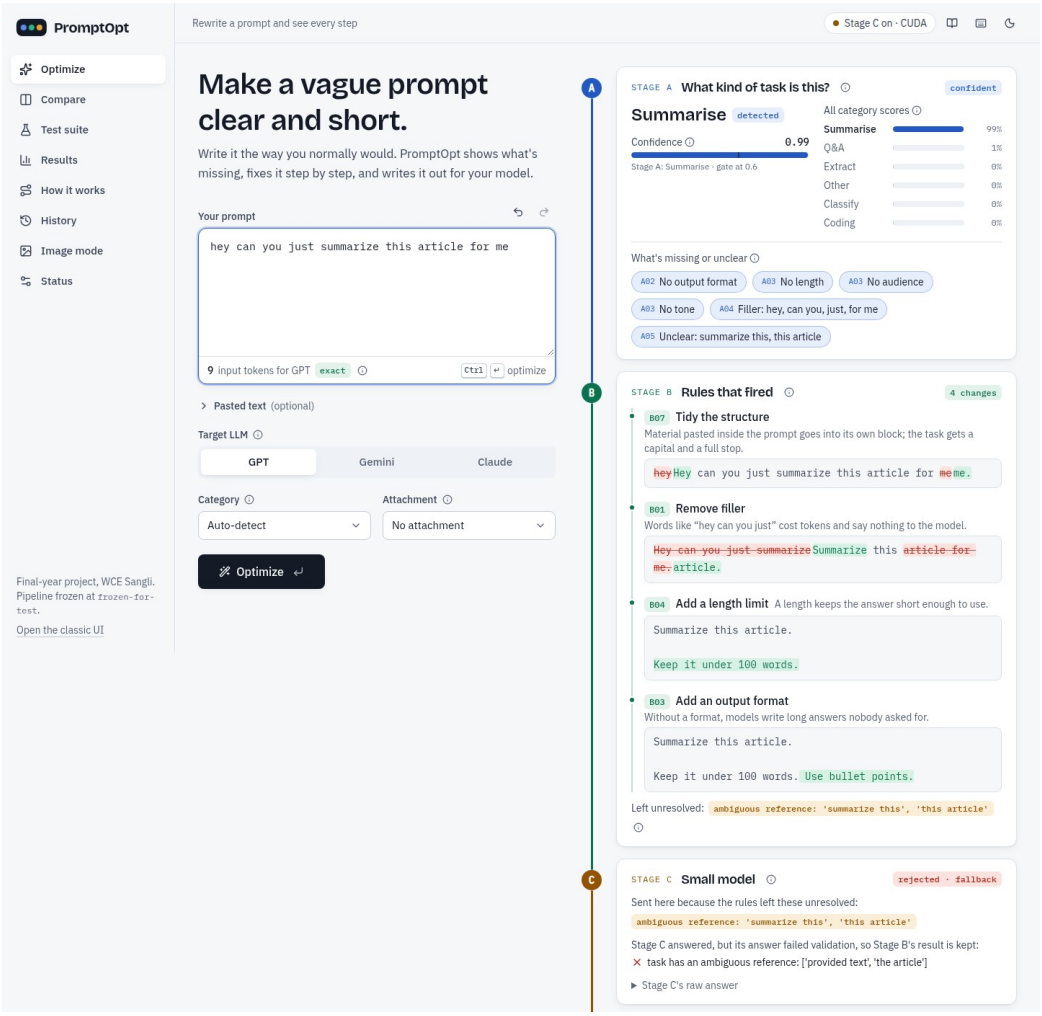


Figure 17: Stage C routed: ambiguous reference sent to Stage C; its answer failed validation, so Stage B's result is kept

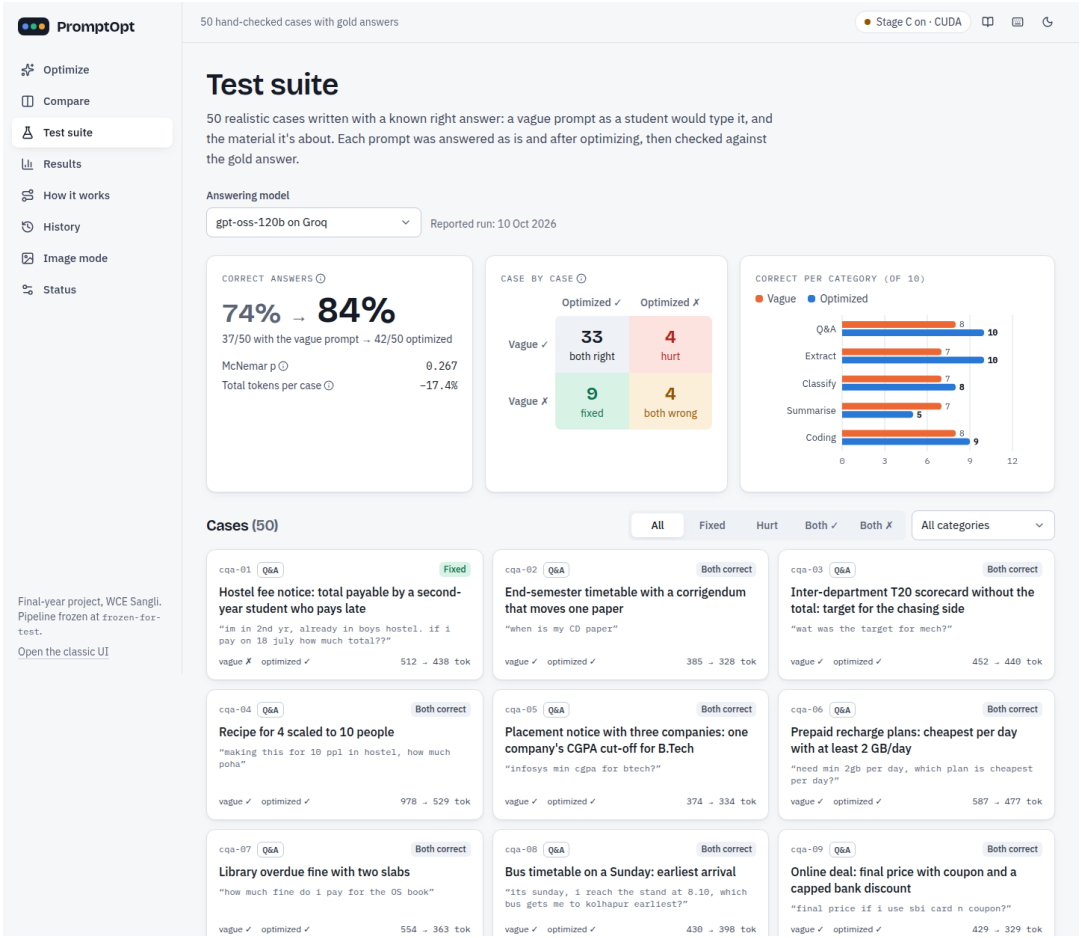


Figure 18: Test suite page: correctness suite summary and case grid

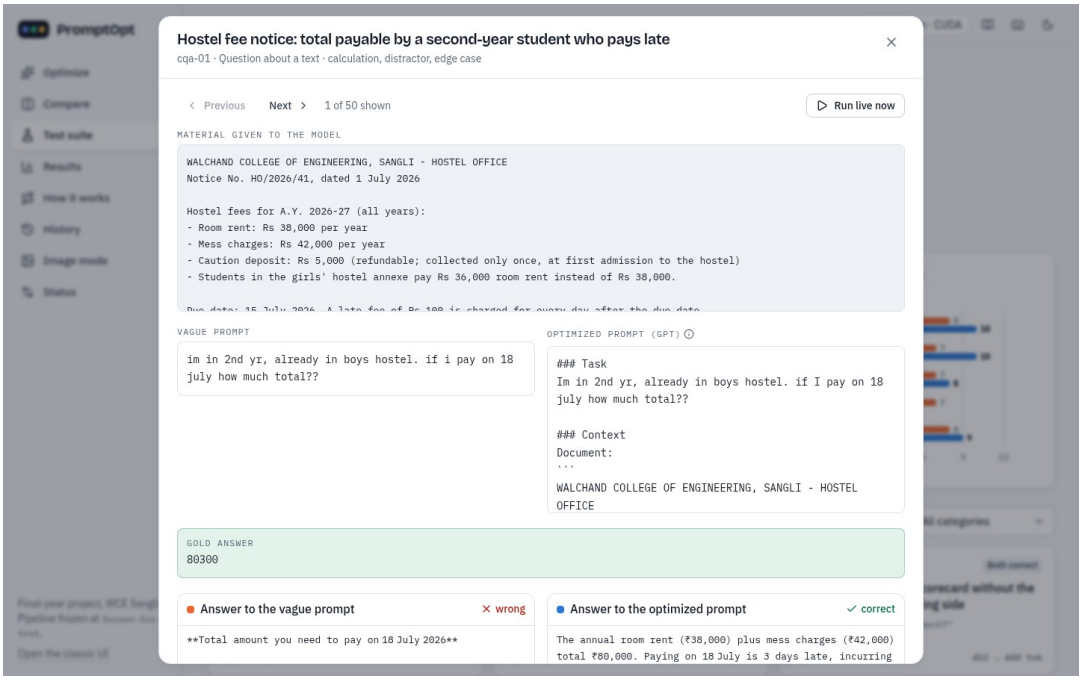


Figure 19: Test suite: one case with gold answer, both prompts and both answers

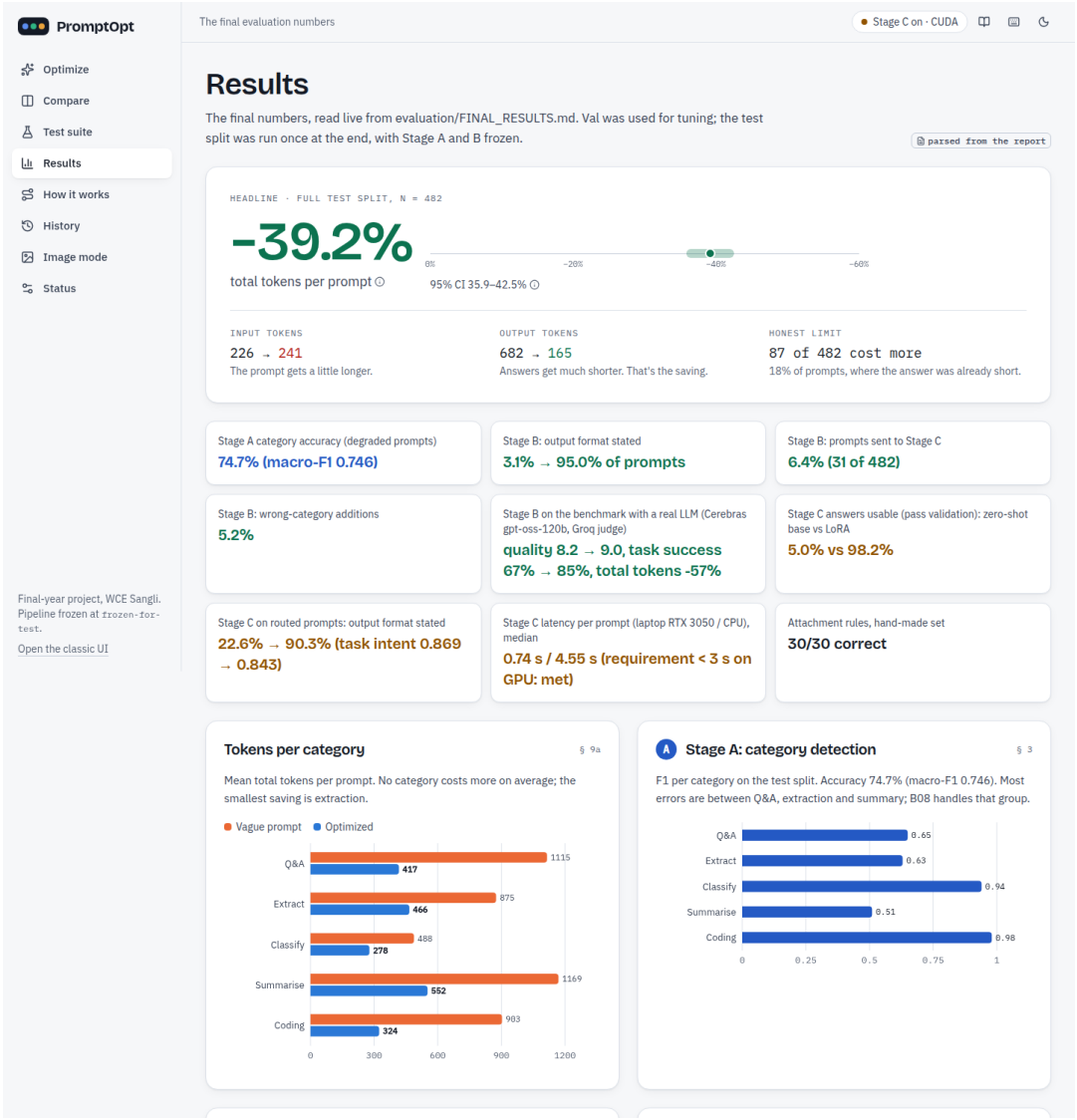


Figure 20: Results page: headline numbers parsed from FINAL_RESULTS.md

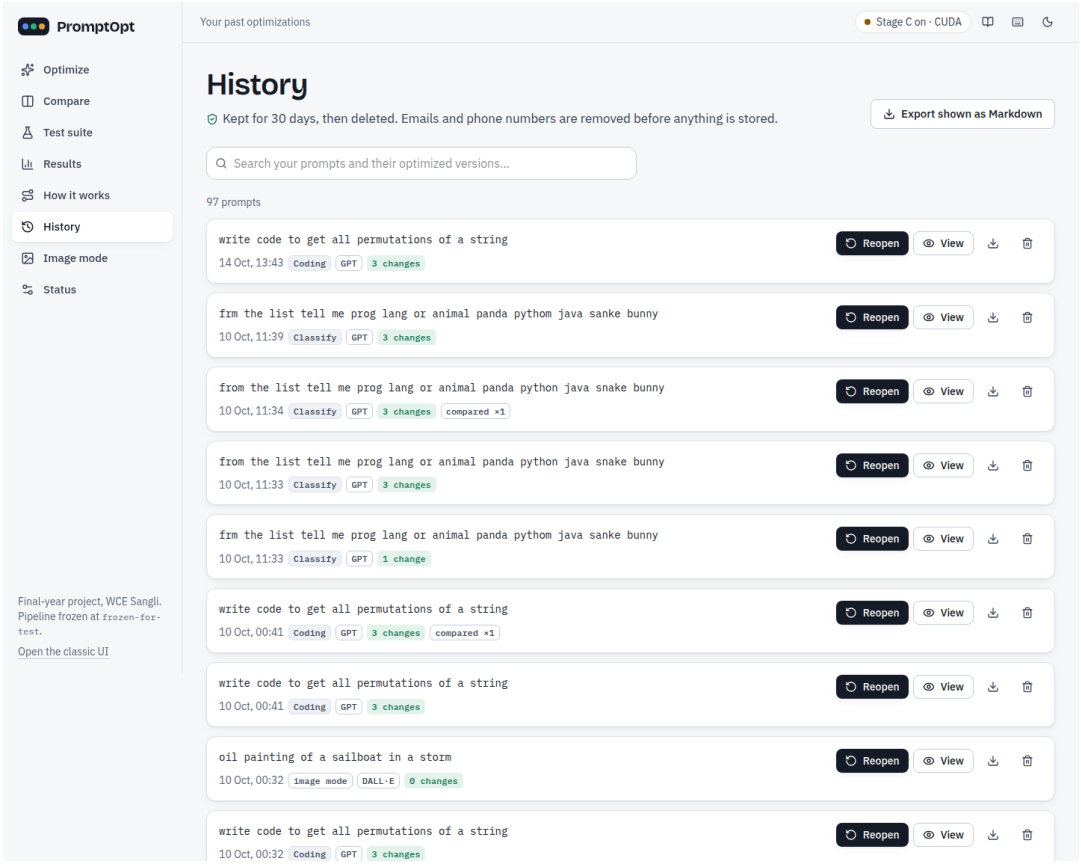


Figure 21: History: search, view and delete past prompts (30-day retention)

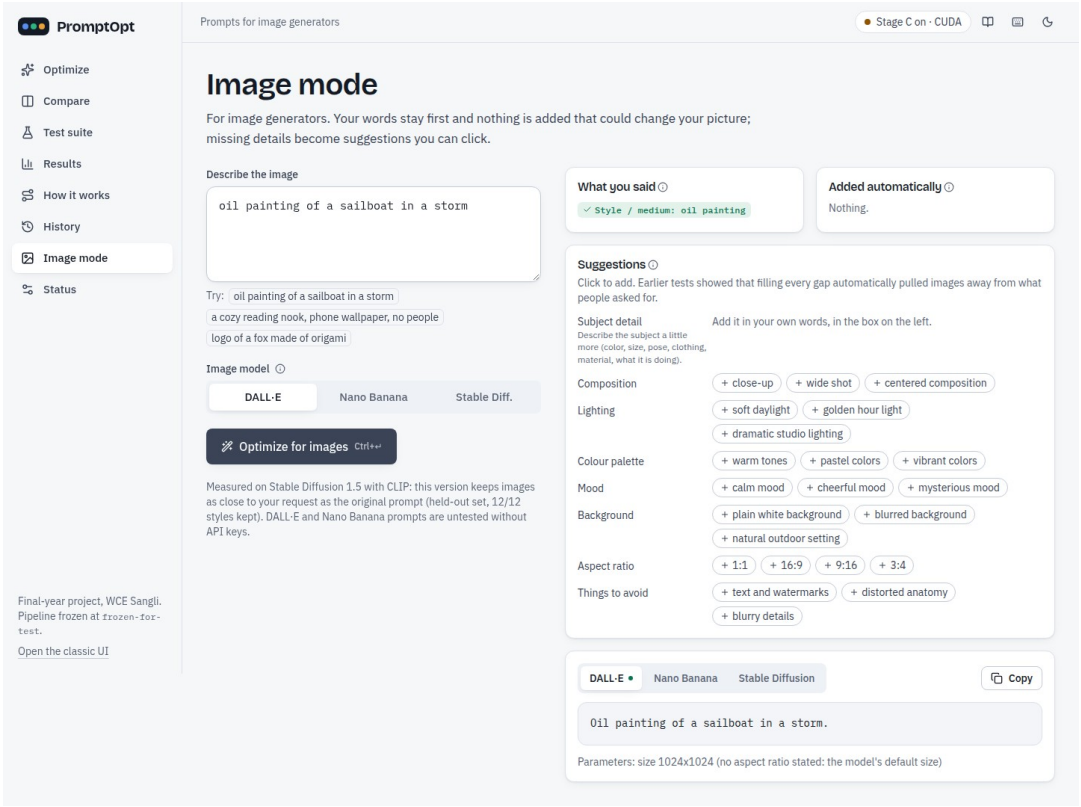


Figure 22: Image mode: user's words kept, missing attributes as clickable suggestions

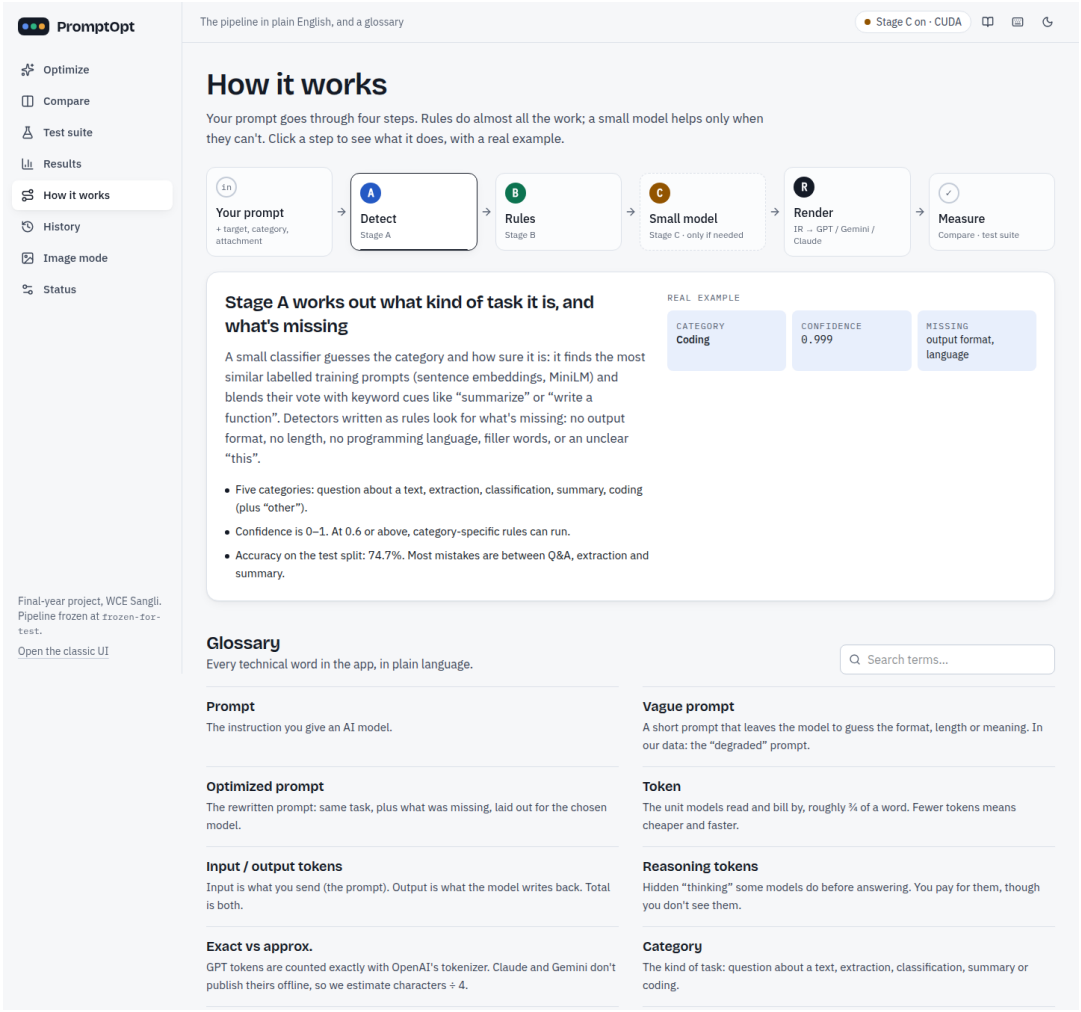


Figure 23: How it works page

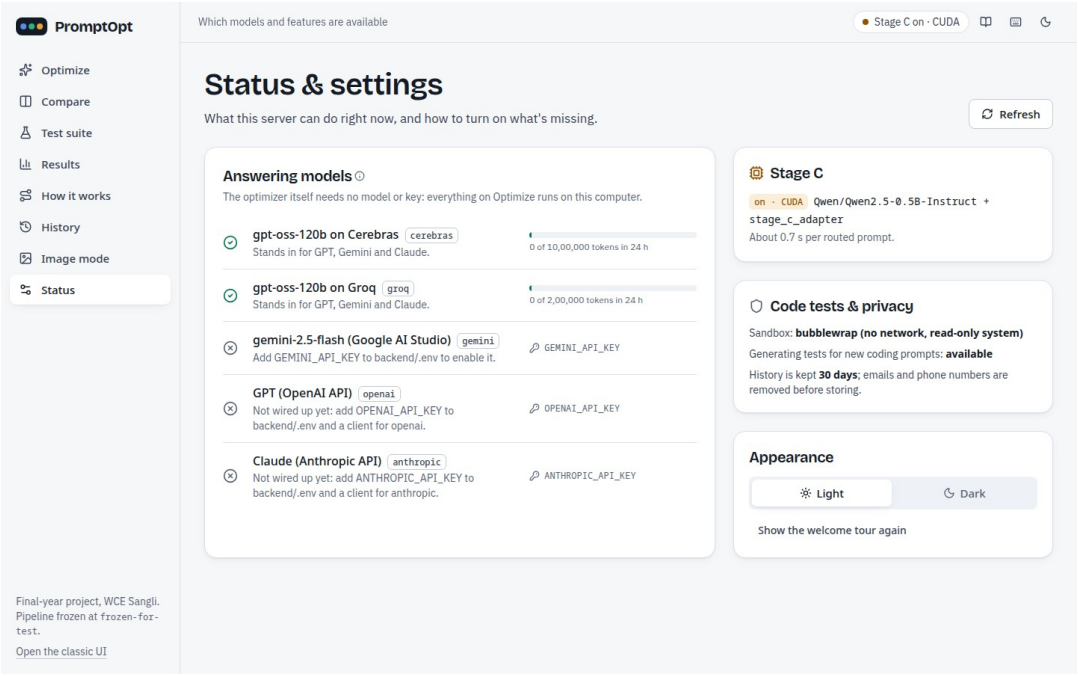


Figure 24: Status page: providers, 24-hour usage, Stage C device, sandbox

f. Screenshot of GitHub project upload

The code is public at https://github.com/piush365/PromptOpt. The screenshots below were captured on 14 October 2026.

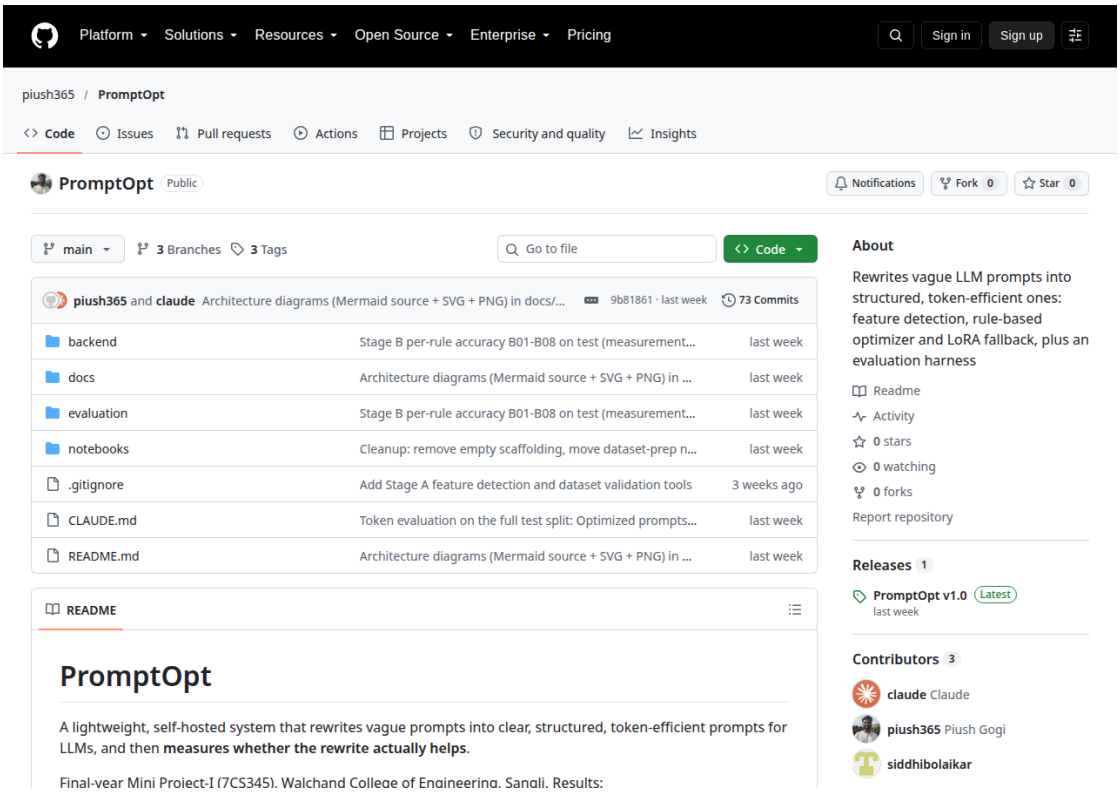


Figure 25: GitHub repository piush365/PromptOpt

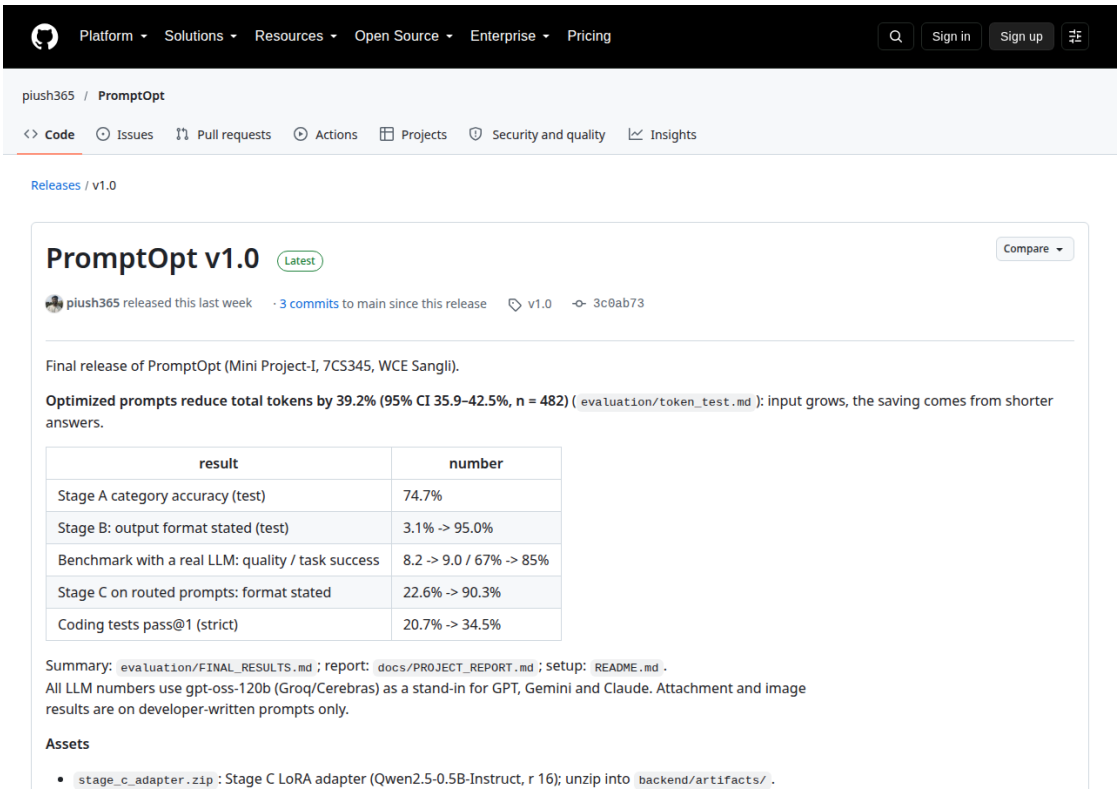


Figure 26: GitHub release v1.0 with the Stage C adapter and Stage A index as assets

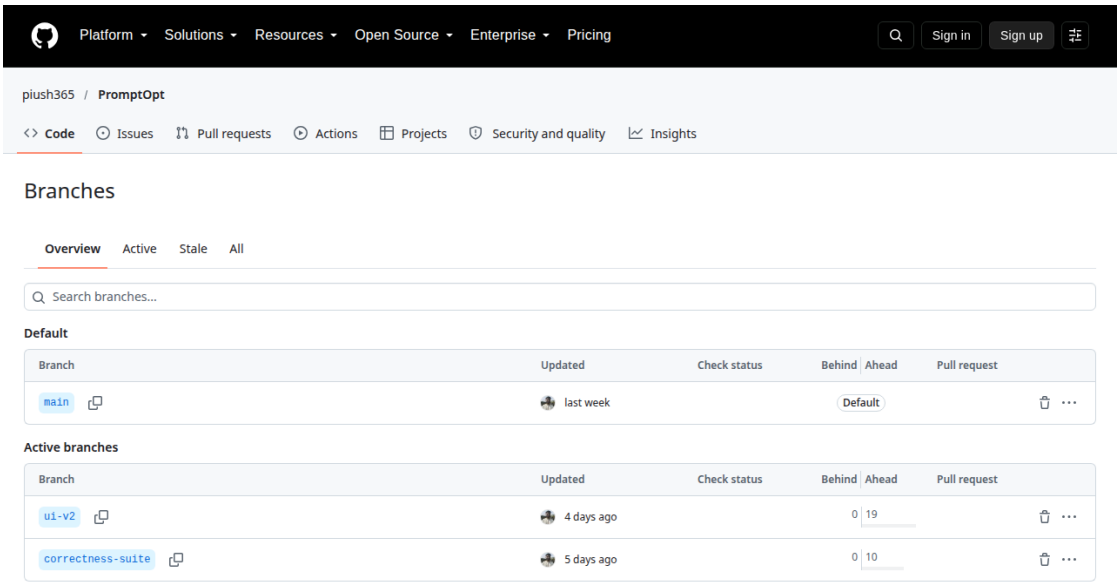


Figure 27: Branches: main (default), ui-v2, correctness-suite

g. Use of Git and GitHub

Table 14: Git and GitHub usage (from git and the GitHub repository, 14 October 2026)

Item	Value
Repository	github.com/piush365/PromptOpt (public)
Commits (all branches)	92 (main: 73; ui-v2: 19 ahead of main; correctness-suite: 10 ahead of main)
Commit period	13 Aug 2026 - 10 Oct 2026; 7 in Aug, 38 in Sep, 47 in Oct
Branches	main (default), ui-v2 (React UI v2), correctness-suite
Tags	frozen-for-test (27 Sep), final-for-test (2 Oct), v1.0 (8 Oct), pre-ui-v2 (9 Oct)
Release	PromptOpt v1.0 (7 Oct 2026): Stage C adapter, Stage A category index, Stage C data manifest
Commit authors in git	piush365 (89), Piush Gogi (2), siddhibolaikar (1)
Work tracking	GitHub Issues, Pull Requests and Projects were not used; work was tracked with the roadmap and phase tables in CLAUDE.md, tags for the frozen/final runs, and a release

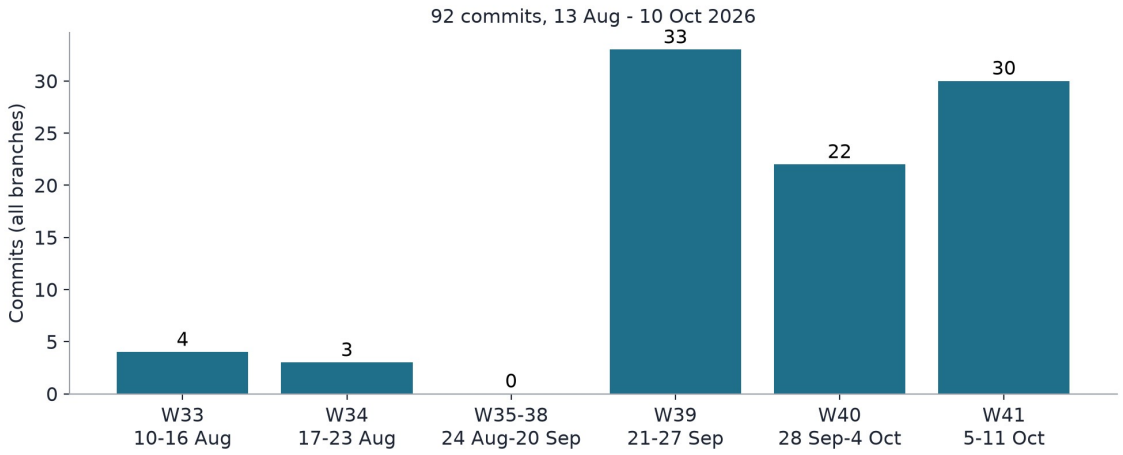


Figure 28: Commit timeline per ISO week (all branches)

Tags were used as evaluation checkpoints: frozen-for-test marks the code that produced the final test numbers; after it, no Stage A/B rule, threshold or judge was changed, and the freeze check proves it.

h. Challenges faced and how we solved them

Table 15: Challenges and solutions

Challenge	What happened	How we solved it
Dataset quality, LLM-generation bugs	LLM-written optimized prompts dropped data from the instruction (e.g. the item list in 18 of 41 classification rows of one revision) or changed the task	Automatic checks + deterministic repair (v1.1: 349 optimized + 20 degraded prompts repaired, 75 subjects restored, every change logged); human rating; LLM-assisted filter
Rater kappa paradox	Raters agreed on 89-94% of rows, but Fleiss' kappa was near 0 (0.065) because 96-98% of answers were "yes"	Reported Gwet's AC1 (0.92-0.96) next to kappa and explained the paradox; kappa target (> 0.6) honestly reported as not met
API quota limits	Free tiers: Groq 200,000 tokens/day per model, Cerebras 150 requests/hour	Rolling 24-hour usage ledger, pacing, 70% budget rule, resumable runs that span two days
Stage C fallback / hallucination	Zero-shot base model passed validation in only 5.0% of cases; a model can invent fields or keep "this"	LoRA fine-tuning (98.2% pass), strict JSON contract with exact keys, validation with Stage A detectors, fallback to Stage B, routed-only use
Image mode v1 failure	Default keywords ("natural lighting") turned a watercolor request into a photo; CLIP fell ($p = 0.043$)	v2: user's words first, defaults only as clickable suggestions; held-out set committed before v2 code
Label / regex edge cases	"thank you so much" inside a button text, "tell me X or Y" labels, items glued without a colon	Unit tests per edge case; B16 wider label detection added for the app only, freeze check unchanged
4 GB GPU training	Qwen2.5-0.5B fine-tuning on an RTX 3050 laptop GPU; later several Stage C copies filled the GPU under concurrent requests	Micro-batch 1 x 16 accumulation, bf16, gradient checkpointing (51 min); models loaded once under a lock

8. Conclusion

PromptOpt shows that a cheap, explainable, rule-first optimizer can make vague prompts clearer and cheaper: on 482 held-out test prompts optimized prompts reduce total tokens by 39.2% (95% CI 35.9-42.5%), state an output format in 95.0% of cases instead of 3.1%, and on the benchmark raise quality from 8.2 to 9.0 and task success from 67% to 85%. A small LoRA model helps exactly where the rules stop (format stated on routed prompts 22.6% -> 90.3%) and is kept out of everything else. The working prototype (FastAPI + React, ten-table database with one commit per request, Compare, History, image mode) is covered by 798 automated tests and 13 browser tests, and the frozen pipeline is verified byte-identical.

Limitations of proposed solution

- All LLM numbers use gpt-oss-120b on Groq/Cerebras as a stand-in for GPT, Gemini and Claude; Claude/Gemini token counts are approximate.
- Input tokens grow in every category; 87 of 482 test prompts cost more in total because their answer was already short.
- Stage A reaches 74.7% category accuracy; closed_qa / information_extraction / summarization remain hard to separate.
- Correctness suite: 42/50 vs 37/50 is not statistically significant (McNemar p = 0.267) and not reproduced on Cerebras (41 vs 43).
- Stage C is trained on LLM-written targets; the routed test set is small (31 prompts).
- Attachment and image results are on developer-written prompts only; coding tests cover Python only (n = 29).
- Kappa target for dataset validation not met (kappa paradox).

Future scope / possible extension to proposed project

- Lean mode for short-answer prompts: add only the output-format line, or nothing, when the expected answer is short.
- Route label-set and typo-heavy prompts to Stage C; spelling normalisation that the user confirms.
- Blind user tests with prompts written by people outside the team (attachment and image blind sets are ready).
- Compare and the full evaluation on real GPT, Claude and Gemini once paid API keys are available.
- A larger correctness suite (more cases per category, more providers) to reach statistical significance.
- Deployment beyond the LAN (PostgreSQL, HTTPS, login); coding tests beyond Python.

9. References

[1] K. Ramnath et al., "A Systematic Survey of Automatic Prompt Optimization Techniques," in Proc. EMNLP, 2025.

[2] L. Ngweta et al., "Towards LLMs Robustness to Changes in Prompt Format Styles," in Proc. NAACL Student Research Workshop, 2025.

[3] T. Zehle et al., "CAPO: Cost-Aware Prompt Optimization," in Proc. AutoML Conference, 2025.

[4] Y. Zhou et al., "Large Language Models Are Human-Level Prompt Engineers," in Proc. ICLR, 2023.

[5] C. Yang et al., "Large Language Models as Optimizers," in Proc. ICLR, 2024.

[6] H. Jiang et al., "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," in Proc. EMNLP, 2023.

[7] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," in Proc. ICLR, 2022.

[8] M. Conover et al., "Free Dolly: Introducing the World's First Truly Open Instruction-Tuned LLM," Databricks, 2023 (databricks-dolly-15k).

[9] S. Chaudhary, "Code Alpaca: An Instruction-following LLaMA Model for Code Generation," GitHub repository, 2023.

[10] Qwen Team, "Qwen2.5 Technical Report," arXiv:2412.15115, 2024.

[11] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP-IJCNLP, 2019.

[12] K. L. Gwet, "Computing inter-rater reliability and its variance in the presence of high agreement," British Journal of Mathematical and Statistical Psychology, vol. 61, no. 1, pp. 29-48, 2008.

[13] A. R. Feinstein and D. V. Cicchetti, "High agreement but low kappa: I. The problems of two paradoxes," Journal of Clinical Epidemiology, vol. 43, no. 6, pp. 543-549, 1990.

[14] A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," in Proc. ICML, 2021.

[15] R. Rombach et al., "High-Resolution Image Synthesis with Latent Diffusion Models," in Proc. CVPR, 2022.

[16] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.

10. Scholarly contribution

It includes

A. Research paper published/accepted details.

None yet. [PLACEHOLDER: add if a paper is submitted or accepted]

B. Patent filing/Copyright.

None. [PLACEHOLDER: add if filed]

C. Project presentation in any event, conference presentation

LA-1 review presentation (Mini Project-I, 2026-27); LA-2 review on 15 October 2026. [PLACEHOLDER: add any other event]

D. Sponsorship Letter

Not applicable (no sponsor). The project uses only free tiers (Groq, Cerebras) and a personal laptop GPU.

11. Customer Connect

i. Letter of appreciation from respective customer/

Not available. [PLACEHOLDER: attach if received]

ii. Customer feedback on application,

Survey for the LA-1 review (students, developers, small teams) [PLACEHOLDER: number of respondents and survey date]: 72% said they run out of tokens very quickly, 58% need multiple prompts to get one output, 46% notice a big drop in quality as tokens run low and 31% have to stop their work due to token limits. Asked how often they must ask an AI tool more than once to get the answer they want: very often 20%, often 30%, sometimes 25%, rarely 15%, never 10%. Our own users of the data: three team raters (330 rows) and one faculty reviewer (19 of 20 rows accepted) validated the dataset; the team confirmed all 50 correctness-suite gold answers.

iii. email from customer,

Not available. [PLACEHOLDER]

iv. audio/video feedback from customer,

Not available. [PLACEHOLDER]